Implementation and Evaluation of the 1990 Simplified Jordan Sorting Algorithm

Master's Thesis

submitted by

Author Name

Matriculation number: Matriculation Number

Supervisor:
Supervisor Name

Second examiner:
Second Examiner Name

Submission date:
Submission Date

Freie Universität Berlin
Department of Mathematics and Computer Science
Institute of Computer Science
Theoretical Computer Science

Abstract

Jordan sorting recovers the coordinate order of intersections along a line from the order in which a simple curve encounters them. Historical linear-time algorithms solve this problem through an incremental sorting procedure combined with specialized dynamic-list structures. This thesis studies whether the control flow of the 1990 Simplified Jordan Sorting algorithm can be reconstructed as an executable, auditable implementation when those theoretical structures are replaced by ordinary Python lists.

The reconstruction maintains its own partial sorted order, upper and lower pair families, and sibling-list ownership while executing the paper-facing Step 1/2/3 procedure. For oracle-certified valid inputs, the paper core recovers its result from the maintained partial order and does not consume the oracle's sorted output. The revised core propagates update errors without rollback and requires failed state to be discarded. Untimed checked-state audits, deterministic replay, and fixed policies distinguish diagnostics from the pre-certified `minimal` call. The archived formal experiment evaluated the earlier rollback-enabled version on 60 controlled cases at five sizes from $n = 32$ to $n = 512$, alongside Python sort and the complete oracle-backed reference pipeline.

All 3600 measured rows have correct output and no recorded error, and all 60 untimed checked-state audits pass. The median exact-case paper/reference runtime ratio decreases from 3.226 at $n = 32$ to 0.567 at $n = 512$; it is above one through $n = 128$ and below one at $n = 256$ and $n = 512$. Python sort has the lowest median call time at every tested size. These ratios are pipeline-scope comparisons: the reference measurement includes its complete oracle-backed pipeline, whereas the paper measurement contains only the pre-certified `minimal` sorting call, with oracle certification and checked diagnostics outside timing. They are therefore not like-for-like end-to-end speedups.

The contribution combines an independently recovering reconstruction with reproducible historical measurements. A later real finger-tree backend has bounded qualification and calibration, but no formal performance results or completed general proof review. Recognition is not measured. The ordinary backend incurs quadratic insertion-copying work on increasing inputs without splits; no tight overall bound is claimed. Historical timings measure neither the revised core nor the benefit of removing rollback.

Contents

Abstract . . . . . **i**

List of Figures . . . . . **vii**

List of Tables . . . . . **ix**

1. Introduction **1**

1.1. Motivation 1

1.2. Problem and Scope 2

1.3. Research Questions 2

1.4. Contributions 3

1.5. Headline Findings 4

1.6. Thesis Structure 4

2. Background **5**

2.1. Jordan Sequences and the Sorting Problem 5

2.2. The Structural Restriction Behind Jordan Sorting 6

2.3. The 1986 Linear-Time Algorithm 7

2.4. The 1990 Simplification 8

2.5. Related Work and Positioning 8

2.5.1. Ordered Lists and Finger Search 8

2.5.2. Jordan Sorting and Geometric Applications 9

2.6. Position and Scope of This Thesis 9

3. Algorithm Reconstruction **11**

3.1. Purpose and Reconstruction Boundary 11

3.2. Pairs, Intervals, and the Two Families 12

3.2.1. Families and Sibling Lists 12

3.3. Maintained State and Prefix Invariant 13

3.4. Initialization 13

3.5. Boundary Selection . . . 14
3.5.1. Step 1: Predecessor-Side Boundary . . . 14
3.5.2. Step 2: Successor-Side Boundary . . . 14
3.6. Step 3: Increasing Orientation . . . 14
3.6.1. Step 3(a): Insert the New Pair . . . 15
3.6.2. Step 3(b): Split Off Newly Enclosed Children . . . 15
3.6.3. Step 3(c): Insert the New Point . . . 15
3.7. Step 3: Decreasing Orientation . . . 15
3.8. Executable Anchor Clarifications . . . 16
3.8.1. Curve Endpoint Versus Geometric Endpoint . . . 16
3.8.2. Odd-Index z_1 Output Anchor . . . 17
3.9. Ownership Transfer and Failure Semantics . . . 17
3.10. End-to-End Procedure . . . 19
3.11. Correctness and Complexity Boundaries . . . 21

4. Implementation . . . 23
4.1. Implementation Scope and Versions . . . 23
4.2. Data Representation . . . 23
4.2.1. Partial Sorted Order . . . 24
4.2.2. Sibling Lists and Ownership . . . 24
4.3. Core–Backend Interface and Update Semantics . . . 24
4.3.1. Initialization, Boundaries and Stage State . . . 24
4.3.2. Pair Insertion and Failure Propagation . . . 25
4.3.3. Split Plans and Ownership Transfer . . . 25
4.3.4. Point Insertion and Completion . . . 26
4.4. Certification, Diagnostics and Replay . . . 26
4.5. Execution Policies and Observation Costs . . . 27
4.6. Actual Costs and Complexity Boundary . . . 28

5. Experimental Method . . . 29
5.1. Study Scope and Version Boundary . . . 29
5.2. Current Inputs and Qualification . . . 29
5.2.1. Historical Input Protocol . . . 30
5.3. Compared Implementations and Call Scopes . . . 30
5.3.1. Python Sort . . . 30
5.3.2. Reference Pipeline . . . 30
5.3.3. Historical Ordinary-List Core . . . 31

5.4. Timing, Calibration and Scheduling . . . 31
5.4.1. Scheduling and Noise . . . 31
5.4.2. Recorded Environment . . . 32
5.5. Aggregation and Statistical Units . . . 32
5.6. Evidence Binding and Acceptance . . . 32
5.6.1. Fail-Closed Validation . . . 34
5.7. Interpretation Boundary . . . 34

6. Results . . . 35
6.1. Evidence Status and Correctness . . . 35
6.2. Runtime by Input Size . . . 35
6.3. Matched-Case Runtime Ratios . . . 37
6.4. Family-Specific Ratios . . . 38
6.5. Runtime Variability . . . 39
6.6. Measured Calls and Pipeline Wall-Clock . . . 39
6.7. Pilot–Formal Directional Replication . . . 40
6.8. Exploratory Relationships . . . 40
6.8.1. Structure and Runtime . . . 41
6.8.2. Checked Counters and Paper Runtime . . . 41
6.9. Results Boundary . . . 42

7. Limitations . . . 43
7.1. Scope of the Limitations . . . 43
7.2. Implementation Version and Failure Contract . . . 43
7.3. Paper Reconstruction and Backend Fidelity . . . 43
7.4. Tested Sizes and Asymptotic Interpretation . . . 44
7.5. Non-Equivalent Timing Scopes . . . 44
7.6. Output Provenance and Recognition . . . 45
7.7. Case Construction and Generality . . . 45
7.8. Execution Environment and Cross-Experiment Replication . . . 45
7.9. Exploratory Structure and Counter Relationships . . . 46
7.10. Runtime Variability and Statistical Reach . . . 46
7.11. Consequences for the Thesis Conclusions . . . 46

8. Conclusion . . . 47
8.1. Summary . . . 47
8.2. Answers to the Research Questions . . . 48
8.2.1. RQ1: Executable Reconstruction and Correctness . . . 48

8.2.2. RQ2: Observed Runtime Behavior 48
8.2.3. RQ3: Structure and Implementation Cost 48
8.3. Contributions in Context 49
8.4. Future Work 49
8.5. Closing Perspective 50

A. Claim Traceability 51
A.1. Claim Groups and Thesis Locations 51
A.2. Implementation Revision and Evidence Scope 52
A.3. Archived Evidence Bindings 53
A.4. Interpretation Boundaries 53

Bibliography 55

List of Figures

2.1. Jordan sorting from curve order to axis order. 6

3.1. Step 3(c) output-anchor correction for z_1. 18
3.2. Running example of the ordinary-list reconstruction. 20

4.1. Pair families and sibling-list ownership. 25

5.1. Historical formal pipeline and timing scopes. 33

6.1. Historical baseline. Median runtime by input size in the formal experiment. The logarithmic vertical axis displays equal-weighted exact-case medians under the distinct timed scopes defined in section 5.3. 36
6.2. Historical baseline. Case-median runtime ratios by input size in the formal experiment. Ratios are formed within exact cases before equal-weight aggregation; the dashed line marks a ratio of one. 37

List of Tables

3.1. Increasing and decreasing Step 3 roles in the executable reconstruction. 16

6.1. Historical baseline. Median exact-case runtime by input size. Values are milliseconds; each size contains twelve exact cases. 36
6.2. Historical baseline. Median exact-case paper/reference ratio by input size. 37
6.3. Historical baseline. Overall median exact-case runtime ratios across all sixty cases. 38
6.4. Historical baseline. Median exact-case paper/reference ratio by family and input size. 38
6.5. Historical baseline. Relative-IQR summary over exact case-algorithm cells. 39
6.6. Historical baseline. Captured measured-call time by algorithm. 39
6.7. Historical baseline. Paper/reference size medians in the pilot and formal experiments. 40
6.8. Historical baseline. Within-size Spearman coefficients between runtime and two structural measures. 41
6.9. Historical within-size Spearman coefficients between logical checked counters and rollback-enabled minimal paper runtime (source `98868b1`). 41

A.1. Controlled claim groups and their principal locations in the thesis. 51
A.2. Archived run manifests used by the thesis analysis. 53

1. Introduction

1.1. Motivation

Sorting normally starts with an arbitrary sequence of keys. Jordan sorting starts with more information: the keys are coordinates of intersections encountered while traversing a simple plane curve. Their input order along the curve and their desired order along a line differ, but the absence of self-intersections restricts how consecutive curve segments can be nested. That geometric restriction permits specialized recognition and sorting algorithms with a theoretical linear-time bound (Hoffmann et al. 1986; Fung et al. 1990).

Jordan sorting lies at the intersection of computational geometry, algorithm design, and data-structure engineering. More generally, computational geometry studies how geometric restrictions can be converted into algorithmic structure and supported by purpose-built data structures (Berg et al. 2008). This setting makes Jordan sorting a useful case study in the difference between an abstract algorithmic result and the costs of a concrete representation.

The published bound, however, combines an algorithmic control flow with specialized dynamic-list representations. The 1986 algorithm uses level-linked search trees, while the 1990 simplification relies on heterogeneous finger trees to support local list operations efficiently. A direct implementation with ordinary arrays or lists can preserve the high-level algorithm while having different operation costs. Moreover, the 1990 presentation is concise: translating its incremental steps into executable code requires explicit choices about pair orientation, sibling-list ownership, geometric endpoints, and the exceptional role of the first point.

This thesis studies that implementation gap. It reconstructs the 1990 Step 1/2/3 sorting procedure as an executable Python core, maintains its own partial sorted order and family state, and evaluates the resulting ordinary-list implementation on controlled, oracle-certified valid inputs. The aim is not to reproduce the historical linear-time data structure or to infer asymptotic complexity from runtime measurements. It is to make the paper-facing control flow concrete, auditable, and empirically observable while keeping the boundary between theorem, implementation, and experiment explicit.

This combination of reconstruction, implementation, and controlled evaluation follows the algorithm-engineering view that design, analysis, implementation, and experimentation are related but distinct parts of algorithmic research (Sanders 2009). The experimental component is likewise treated as a source of bounded evidence about implemented algorithms, not as a replacement for theoretical analysis (McGeoch 2012).

1.2. Problem and Scope

For a Jordan sequence $z_1, \ldots, z_n$, the computational task considered here is to recover increasing coordinate order from curve-traversal order. The geometric model and its nested upper and lower pair families are introduced in chapter 2; the executable reconstruction is specified in chapter 3.

The thesis considers two deliberately different research implementations. The *simplified reference pipeline* performs oracle validation, constructs structural information, and returns oracle-derived sorted output. The *paper ordinary-list core* instead accepts a pre-certified valid input, executes the reconstructed incremental procedure, and recovers its output from the maintained partial order. It does not consume the oracle's sorted result. An outer wrapper may certify the input before calling the core, and the experiment may compare its return value with the oracle afterward; both operations are outside the timed paper call. This provenance boundary is formalized in section 4.4.

The historical empirical scope is valid-input sorting. Recognition of arbitrary or invalid sequences, error correction, polygon clipping, and the historical theoretical backends are not evaluated. Sibling lists use ordinary Python lists, so scans, slicing, copying, and ownership rebinding remain part of the implemented cost. Consequently, the historical linear-time theorem does not transfer to this implementation.

The current revision documents ordinary-list and real finger-tree backends that terminate on update errors without rollback. The tree has bounded qualification and calibration, but no formal matched samples. Performance evidence reported here belongs to its earlier rollback-enabled version. The two versions share the reconstructed sorting structure, but the archived measurements do not establish the revised implementation's runtime or the benefit of removing recovery work. Section 4.1 makes this distinction explicit.

1.3. Research Questions

Within those boundaries, the thesis addresses three questions:

RQ1: Executable reconstruction and correctness. Can an ordinary-list reconstruction of the 1990 incremental sorting procedure independently recover sorted order for oracle-certified valid inputs while maintaining the required partial-order and sibling-family invariants?

RQ2: Observed runtime behavior. Across the pre-specified input sizes and controlled families, how does the pre-certified paper call in `minimal` mode behave relative to Python sort and the complete oracle-backed reference pipeline under their explicitly different timing scopes?

RQ3: Structure and implementation cost. Within the controlled sample, what descriptive relationships appear between structural properties, checked operation counters, and the runtime of the ordinary-list implementation?

The first question is addressed through focused tests, bounded exhaustive validation over oracle-certified cases, checked-state audits, and deterministic replay. The second uses the formal experiment under its pre-specified protocol. The third is exploratory: its associations are descriptive rather than causal or asymptotic.

1.4. Contributions

The work presented in this thesis makes four implementation and empirical contributions.

1. It gives an executable reconstruction of the 1990 Step 1/2/3 control flow, informed by the 1986 algorithm where the later presentation leaves operational details implicit. The reconstruction records its endpoint, orientation, and first-point clarifications rather than presenting them as the only possible reading of the paper.
2. It implements separate partial-order and sibling-family structures with explicit ownership, checked split transitions, local safety checks, and full invariant auditing. Internal update errors terminate the revised sorter, and failed state must be discarded. The paper core recovers output from this maintained state rather than from the oracle-backed reference path.
3. It separates optional observation and complete audit machinery from the timed path through fixed execution policies. Production sorting and checked diagnostics share one core control flow, while deterministic replay and complete state validation provide an untimed audit of recorded state. The `minimal` policy still retains local safety, stage state, and final output recovery.
4. It establishes a reproducible empirical workflow with pre-specified cases, documented timing scopes, archived manifest-bound evidence, and a separate fail-closed evidence-contract validator. The resulting study compares Python sort, the oracle-backed reference pipeline, and the paper ordinary-list call without treating their scopes as interchangeable.

These contributions concern reconstruction, correctness engineering, and empirical characterization. The later heterogeneous finger-tree implementation extends the executable scope, but does not

establish a reviewed linear-time theorem or current formal performance results. Level-linked search trees remain outside the implemented scope.

1.5. Headline Findings

All 3600 measured rows in the formal experiment on the earlier implementation belong to 60 exact cases that each passed one untimed checked-state audit before timing. Every row records an oracle-certified input, correct output, and no recorded error. All 180 case-algorithm summaries are correct and error-free, and all 60 case audits pass. This is empirical correctness evidence for the pre-specified cases, not a proof for every Jordan sequence.

The median exact-case paper/reference runtime ratio decreases across the five tested sizes, from 3.226 at $n = 32$ to 0.567 at $n = 512$. It is above one through $n = 128$ and below one at $n = 256$ and $n = 512$. This ratio compares two different timed pipelines: the reference value includes its complete oracle-backed reference work, whereas the paper value includes only the pre-certified `minimal` sorting call, with certification and checked diagnostics outside timing. The observation is therefore a pipeline-scope comparison over five tested sizes, not a like-for-like end-to-end speedup or evidence of asymptotic complexity. Python sort has the lowest median call time at every tested size. Full numerical results and variability information are reported in chapter 6, with interpretation limits collected in chapter 7.

1.6. Thesis Structure

Chapter 2 defines Jordan sorting, explains the nested family structure, and places the 1986 and 1990 algorithms in context. Chapter 3 develops the implementation-facing reconstruction of the incremental procedure. Chapter 4 describes the concrete partial-order, sibling-list, audit, and execution-policy components. Chapter 5 specifies the pre-specified valid-input experiment, timing boundaries, statistical units, and evidence contract. Chapter 6 presents correctness, runtime, variability, replication, and exploratory findings. Chapter 7 states the backend, timing, sampling, and inferential boundaries. Finally, chapter 8 summarizes the answers to the research questions and identifies work that remains outside the thesis.

2. Background

2.1. Jordan Sequences and the Sorting Problem

Let C be a simple plane curve that crosses the x-axis wherever it meets the axis. Listing the abscissas of the intersection points in the order in which they occur along C gives a sequence

$$z_1, z_2, \dots, z_n. \tag{2.1}$$

Such a realizable sequence is called a *Jordan sequence*. The Jordan sorting problem is to recover the order of the same points along the x-axis, that is, their increasing coordinate order, from their order along the curve (Hoffmann et al. 1986, p. 170). The 1990 presentation uses the term Jordan curve for both open and closed simple curves; the property needed here is the absence of self-intersections (Fung et al. 1990, pp. 85–86).

This formulation gives two distinct orders on one set of points:

Curve order is the input order $z_1, \dots, z_n$ obtained by traversing C.
Axis order is the output order obtained by moving from left to right along the x-axis.

The values are therefore not arbitrary keys presented in an arbitrary permutation. They carry a geometric realizability constraint inherited from the simplicity of C. The implementation studied in this thesis specializes this model to distinct, mutually comparable coordinate values, as stated in section 4.2.

The problem arose from geometric applications rather than from a desire to replace general-purpose sorting. The 1986 paper relates it to ordering the intersections of a simple polygon with a line and to ordering intersections of two Jordan curves. The 1990 paper develops polygon clipping as its main application and also notes the role of Jordan sorting in simple-polygon triangulation (Hoffmann et al. 1986, pp. 170–171; Fung et al. 1990, pp. 85–86). In these settings, intersection points can be obtained naturally in boundary or curve order, while the next geometric stage requires their order along another line or boundary.

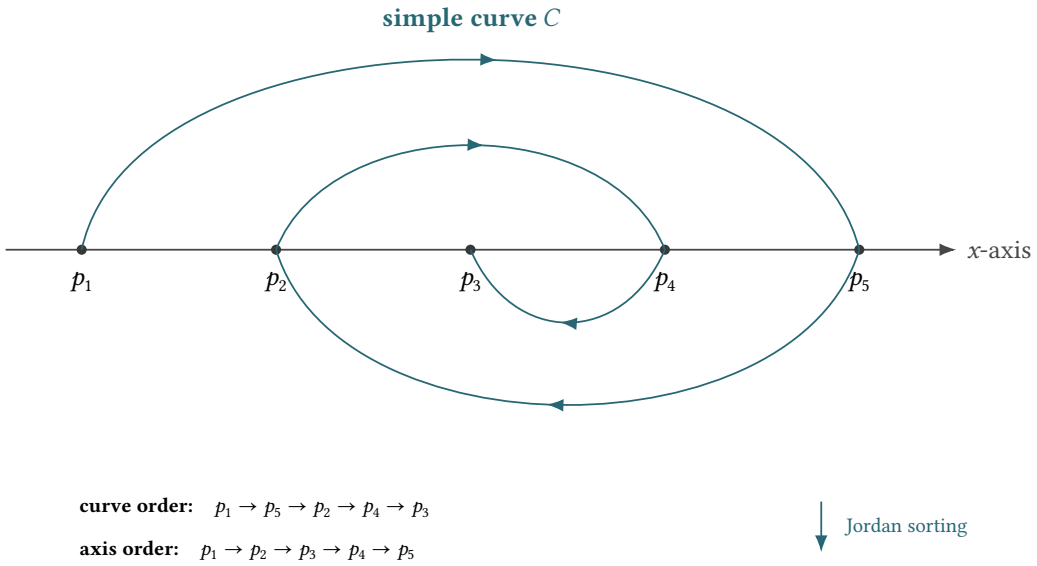


Figure 2.1.: Illustration of the Jordan sorting problem. The simple curve encounters the five intersections in curve order $(p_1, p_5, p_2, p_4, p_3)$, whereas their axis order is $(p_1, p_2, p_3, p_4, p_5)$. Jordan sorting recovers the axis order from the curve order.

2.2. The Structural Restriction Behind Jordan Sorting

Assume, after reflection about the x-axis if necessary, that the first arc lies above the axis. Consecutive intersections delimit alternating arcs above and below it. The pairs

$$\{z_1, z_2\}, \{z_3, z_4\}, \ldots \tag{2.2}$$

belong to the upper side, while

$$\{z_2, z_3\}, \{z_4, z_5\}, \ldots \tag{2.3}$$

belong to the lower side. Each pair determines the open interval between its two coordinates on the axis.

Two intervals from the same side cannot cross. If one endpoint of one such pair lies strictly inside another pair's interval, then its other endpoint must lie there as well; otherwise the corresponding arcs would intersect. Consequently, same-side intervals are disjoint or nested. Replacing the smaller endpoint by a left parenthesis and the larger by a right parenthesis therefore produces one properly nested parenthesis system for each side (Hoffmann et al. 1986, pp. 172–173).

The containment relation gives an ordered forest for each parity family. A pair's parent is its smallest same-family enclosing pair, and siblings are ordered by their left endpoints. Adding a dummy pair

spanning $(-\infty, +\infty)$ turns each forest into a rooted ordered tree. These are the upper and lower family trees used by both historical algorithms (Hoffmann et al. 1986, p. 173; Fung et al. 1990, p. 87). The full notation and executable pair model used in this thesis are developed in section 3.2; here the important point is that simplicity turns a geometric constraint into nested ordered structure.

This structure is what makes specialized algorithms possible for valid inputs. While processing points in curve order, the algorithms maintain both a sorted list of the processed coordinates and the two family trees. Inserting the next pair into its family identifies a local position for the new coordinate in the sorted list (Hoffmann et al. 1986, pp. 173–175). The remaining difficulty is not an unrestricted search through all processed values, but the efficient maintenance and splitting of ordered family lists.

2.3. The 1986 Linear-Time Algorithm

Hoffmann et al. (1986) gave an algorithm that both recognizes and sorts Jordan sequences in $O(n)$ time. It processes the sequence from left to right in curve order, maintaining a doubly linked sorted list and the upper and lower family trees. When a new interval would violate the required nesting, the recognition algorithm aborts; if all points are processed, the maintained list is the sorted result (Hoffmann et al. 1986, pp. 173–175).

The central data-structural operation is splitting a family list around the new interval. A circular doubly linked representation permits local insertion but can require a scan proportional to the smaller side of a split. Across a sequence of splits this yields an $O(n \log n)$ implementation, no better asymptotically than general sorting (Hoffmann et al. 1986, pp. 175–176). The paper obtains its linear bound by replacing those lists with circular level-linked $(2, 4)$ trees.

A level-linked $(2, 4)$ tree stores list items at the leaves of a balanced multiway search tree and connects nodes at equal height. These links support finger searches whose cost depends on the shorter circular distance between the starting item and the target. Together with tree splitting, concatenation, and an amortized analysis, the representation supports the family-list operations within $O(n)$ total time (Hoffmann et al. 1986, pp. 177–183). Thus the 1986 result couples two ideas:

1. reduce Jordan recognition and sorting to incremental family-list manipulation; and
2. realize that manipulation with a specialized level-linked data structure.

The second component is essential to the published linear-time bound.

2.4. The 1990 Simplification

Fung et al. (1990) retained the linear theoretical bound while simplifying both the primary structure and the operations required of it. A locality lemma shows that, after a pair has been added, the relevant children of a family remain in one or two contiguous sibling lists. As a result, the sorting algorithm needs only three abstract list operations:

1. create a singleton list;
2. insert an item at the front or back of a known list; and
3. split a list at a specified item or key.

In particular, the simplified formulation avoids the 1986 operation that extracts a middle section and concatenates the two remaining ends (Fung et al. 1990, pp. 86, 88–90).

The algorithm again processes intersections incrementally. It maintains a sorted list, upper and lower family trees, and sibling lists encoding the local family order. Neighboring processed coordinates locate the relevant same-family boundaries; insertion of the new pair and any necessary sibling split then determine where the new point enters the sorted list. The exact Step 1/2/3 reconstruction used in this thesis, including the symmetric orientation and endpoint clarifications, is the subject of chapter 3 and is not repeated here.

For the theoretical implementation, the 1990 paper proposes heterogeneous finger trees. It states constant amortized time for singleton-list creation and boundary insertion, and a logarithmic-in-the-smaller-side amortized bound for splitting. A potential argument charges the split cost across the operation sequence, giving $O(m)$ total time for m list operations and hence an $O(n)$ Jordan-sorting bound (Fung et al. 1990, pp. 90–91). This is a theorem about the algorithm together with that efficient list representation, not about every concrete implementation of the three-operation interface.

The 1990 paper also states that the sorting method extends to recognition and to the error-correcting Jordan-sorting task used in the polygon-triangulation algorithm of Tarjan and Van Wyk (1988), and that neither extension changes its $O(n)$ bound (Fung et al. 1990, p. 91). The error-correction task belongs to that separate triangulation work; neither it nor recognition is evaluated in this thesis.

2.5. Related Work and Positioning

2.5.1. Ordered Lists and Finger Search

The specialized list backends used for Jordan sorting belong to a broader line of work on ordered-list representations with fingers and locality-sensitive access. Brown and Tarjan (1978) study linear lists

with movable fingers, and Brown and Tarjan (1980) analyze a 2–3-tree representation for sorted lists whose costs reflect locality of reference. Huddleston and Mehlhorn (1982) develop level-linked weak B-trees for arbitrary sequences of sorted-list updates. The level-linked $(2, 4)$ trees used by the 1986 Jordan-sorting algorithm are part of this historical data-structure lineage, rather than a property of Jordan sequences alone.

These works provide context for the historical linear-time theorem, but they do not describe the backend implemented in this thesis. The reconstruction uses ordinary Python lists and therefore retains explicit scan, copy, slice, and rebinding costs.

2.5.2. Jordan Sorting and Geometric Applications

The first symposium publication of the linear-time Jordan-sorting result appeared in 1985 (Hoffmann et al. 1985); the 1986 journal article used throughout this thesis gives the expanded algorithm and data-structure account. The later simplification connected Jordan sorting to polygon clipping (Fung et al. 1990), while the error-correcting task appears in the polygon-triangulation algorithm of Tarjan and Van Wyk (1988). Subsequent work by Bar-Yehuda and Chazelle (1994) used a generalized form of Jordan sorting together with point location and interval trees to triangulate collections of disjoint Jordan polygonal chains.

Other work pursued alternatives to the 1986 and 1990 dynamic-list algorithms. Aurenhammer (1987) related Jordan sorting to convex-hull computation for certain non-simple polygons. Sojka (1997) instead emphasized simpler algorithms intended to be practical at moderate input sizes, including an $O(n \log n)$ worst-case method and an $O(n \log \log n)$ worst-case method. A later average-case study analyzed a new simple algorithm under an explicit random-Jordan-sequence model, identified conditions for linear expected time, and supplemented the analysis with simulation (Sojka 1999). These are distinct algorithmic and analytical directions. This thesis selects the 1990 procedure as an object of executable reconstruction; it neither claims that procedure is the only post-1986 approach nor implements or compares the alternatives in this paragraph.

This literature shows that Jordan sorting participates in a wider computational-geometry research line, rather than being only an isolated sorting problem. The present thesis nevertheless has a narrower target: it reconstructs and evaluates valid-input sorting. It does not implement a new theoretical list structure, polygon clipping, triangulation, error correction, or recognition.

2.6. Position and Scope of This Thesis

This thesis separates three layers that are easy to conflate:

Geometric validity. A candidate sequence is checked against the Jordan-validity model used for certification before the paper core is timed.

Valid-input sorting. The reconstructed paper control flow maintains its own partial order and recovers the final output from that state.

List representation. Sibling lists are implemented with ordinary Python lists rather than either historical theoretical backend.

Accordingly, the empirical study concerns sorting of oracle-certified valid inputs. Recognition remains separate, as specified in section 5.1. The paper core does not consume oracle-sorted output, whereas the simplified reference pipeline deliberately remains oracle-backed; the implementation boundary is detailed in section 4.4.

The ordinary-list representation supports an executable and auditable reconstruction, but it may scan, slice, copy, and rebind list contents. This thesis therefore does not claim to implement the level-linked trees of 1986 or the heterogeneous finger trees of 1990, and it does not transfer the papers' linear-time theorem to the concrete Python backend. The consequences for empirical interpretation are stated in sections 7.3 and 7.5.

3. Algorithm Reconstruction

3.1. Purpose and Reconstruction Boundary

The Jordan sorting problem starts with the intersections of a simple curve and the x-axis in curve order,

$$z_1, z_2, \dots, z_n, \tag{3.1}$$

and asks for the same points in increasing x-coordinate order. The 1990 algorithm of Fung et al. (1990) processes this sequence incrementally while maintaining a sorted list and two ordered family trees. Its presentation is compact: the increasing case is described explicitly, the decreasing case is called symmetric, and some endpoint details are left implicit. This chapter states the executable reconstruction used in the thesis.

Three kinds of statements must therefore remain distinct:

Explicit source rule. A definition or step is stated directly in the 1990 paper, such as the upper/lower family trees, predecessor and successor boundary searches, and increasing Step 3.

Symmetric reconstruction. A decreasing rule is obtained by reflecting the explicit increasing rule: predecessor and successor roles, left and right split sides, child extremes, and insertion direction are exchanged.

Executable clarification. A detail is fixed so that the compact paper description becomes unambiguous code. The geometric child endpoint and the odd-index z_1 output-anchor rule fall into this category.

The last category is not presented as the unique intended wording of an unstated paper detail. It is an implementation-level interpretation supported by the related 1986 algorithm of Hoffmann et al. (1986), by reflected counterexamples, and by bounded exhaustive validation over oracle-certified inputs. The validation evidence checks this reconstruction; it does not replace a topological proof of the source algorithm.

The chapter also separates algorithmic control flow from data-structure complexity. The 1990 paper obtains its linear bound with heterogeneous finger trees. The thesis implementation realizes the same high-level list operations with ordinary Python lists. Consequently, the reconstruction supports an

executable sorting algorithm without a linear-time guarantee for its concrete backend. A source-level quadratic copying lower bound is given in section 4.6.

3.2. Pairs, Intervals, and the Two Families

Input values are assumed to be distinct and mutually comparable. For each $i \geq 2$, define the curve-order pair

$$P_i = \{z_{i-1}, z_i\}. \tag{3.2}$$

The two stored endpoints retain curve order. Their geometric interval is described separately by

$$\ell_i = \min\{z_{i-1}, z_i\}, \qquad r_i = \max\{z_{i-1}, z_i\}. \tag{3.3}$$

A pair P_i encloses a finite point x when

$$\ell_i < x < r_i, \tag{3.4}$$

and it encloses another pair P_j when

$$\ell_i < \ell_j < r_j < r_i. \tag{3.5}$$

After reflecting the curve about the x-axis if necessary, the pairs with even end index belong to the upper family and those with odd end index belong to the lower family:

$$P_2, P_4, P_6, \ldots \in \mathscr{F}^+, \qquad P_3, P_5, P_7, \ldots \in \mathscr{F}^-. \tag{3.6}$$

For pairs in one family, simplicity of the curve implies that their intervals are disjoint or nested. The enclosure relation therefore forms a forest. Each set of siblings is ordered from left to right by increasing ℓ_i.

The reconstruction adds a distinct dummy pair to each family. Each dummy conceptually spans $(-\infty, +\infty)$ and becomes the parent of every pair that has no finite enclosing parent. The two forests are thereby represented as rooted ordered trees, but the upper and lower dummy identities remain separate.

3.2.1. Families and Sibling Lists

Following the locality argument of Fung et al. (1990), the children associated with a family can be represented by one or two ordered sibling lists. The most recently added pair divides the family

region into a left and a right part; children on either side remain contiguous in sibling order. This locality is what reduces the update to three abstract list operations:

1. create a singleton sibling list;
2. insert a pair at a known list boundary; and
3. split a sibling list at the value of the new point.

The algorithm never requires arbitrary insertion into the middle of a sibling list. A split may produce an empty side, but an empty side is not retained as a live sibling list.

3.3. Maintained State and Prefix Invariant

After processing $z_1, \dots, z_k$, the conceptual state contains:

- a partial-order list S_k containing exactly those k points in increasing value order;
- every finite pair $P_2, \dots, P_k$ in its parity family;
- one parent and one sibling-list owner for every finite pair; and
- the two family-specific dummy roots.

The central prefix invariant is

$$S_k = \mathrm{sort}(z_1, \dots, z_k). \tag{3.7}$$

Here `sort` is mathematical notation for the required order, not an operation invoked by the paper core. The maintained list is bounded by identity-based negative- and positive-infinity sentinels so that predecessor and successor queries are defined at both ends.

The family-tree invariant and equation (3.7) are updated together. Pair insertion locates the interval in which z_i belongs, sibling splitting transfers any newly enclosed children, and the resulting family boundary determines the local insertion anchor in S_{i-1}.

3.4. Initialization

Lengths zero, one, and two are handled directly by fixed-size comparison logic. For $n \geq 3$, the incremental state is initialized by ordering z_1, z_2, z_3 with fixed-size comparison logic and creating

$$P_2 = \{z_1, z_2\} \qquad \text{in the upper family,} \tag{3.8}$$

$$P_3 = \{z_2, z_3\} \qquad \text{in the lower family.} \tag{3.9}$$

The upper dummy owns a singleton sibling list containing P_2, and the lower dummy owns one containing P_3. Thus S_3 satisfies equation (3.7), both parity families have a root, and every finite pair has exactly one owner before the main loop begins.

3.5. Boundary Selection

Iteration $i \geq 4$ creates P_i in the family determined by the parity of i. Steps 1 and 2 inspect the immediate neighbors of z_{i-1} in S_{i-1}.

3.5.1. Step 1: Predecessor-Side Boundary

Let v be the predecessor of z_{i-1}. If i is odd and $v = z_1$, replace v by the predecessor of z_1. If the resulting neighbor is the negative sentinel, let A_i be the dummy pair of the current family. Otherwise, let A_i be the already processed same-family pair containing v.

3.5.2. Step 2: Successor-Side Boundary

Let w be the successor of z_{i-1}. If i is odd and $w = z_1$, replace w by the successor of z_1. If the resulting neighbor is the positive sentinel, let B_i be the current family dummy. Otherwise, let B_i be the processed same-family pair containing w.

A processed point can belong to both an upper and a lower pair. The parity of i selects the relevant incident pair without using a global rank map. The special treatment of z_1 is necessary because z_1 belongs to P_2 but to no lower-family pair. These two neighbor adjustments select family-tree boundaries; they are distinct from the output-anchor adjustment introduced in section 3.8.2.

3.6. Step 3: Increasing Orientation

Assume first that $z_{i-1} < z_i$. Let $N = P_i$ denote the new pair. The predecessor-side pair A_i controls where N enters the family tree, while the successor-side pair B_i controls whether an existing sibling list must be split.

3.6.1. Step 3(a): Insert the New Pair

If A_i encloses z_{i-1}, then N becomes its child in a new singleton sibling list. Otherwise, A_i is the last item of its sibling list; N is inserted immediately after A_i and inherits the same parent. In either case, N obtains exactly one parent and one sibling-list owner.

3.6.2. Step 3(b): Split Off Newly Enclosed Children

If B_i encloses z_{i-1}, N acquires no child list. Otherwise, B_i is the first item of an existing sibling list L. Splitting L at z_i produces a left side and a right side. For a valid input, no pair may straddle the split value: both endpoints of each child lie on one side. The left side, whose pairs are newly enclosed by N, becomes a child list of N; the right side remains with the previous owner of L.

3.6.3. Step 3(c): Insert the New Point

If N acquired no children, the base output anchor is z_{i-1}. Otherwise, let C be the rightmost child pair of N and choose the geometric right endpoint $r(C)$ as the base anchor. After the possible odd-index adjustment in section 3.8.2, insert z_i immediately after the output anchor.

3.7. Step 3: Decreasing Orientation

For $z_i < z_{i-1}$, the reconstruction uses the order reflection of the increasing case. This is the executable meaning assigned to the 1990 paper's statement that the other case is symmetric.

Step 3(a) now uses B_i. If B_i encloses z_{i-1}, N becomes its child in a singleton sibling list. Otherwise, B_i is the first item of its list; N is inserted immediately before it and inherits its parent.

Step 3(b) uses A_i. If A_i encloses z_{i-1}, no split is needed. Otherwise, A_i is the last item of its sibling list. The list is split at z_i; the right side is transferred to N, while the left side remains with the old owner.

For Step 3(c), the no-child base anchor is again z_{i-1}. With children, let C be the leftmost child pair of N and use its geometric left endpoint $\ell(C)$. After the reflected odd-index adjustment, insert z_i immediately before the output anchor.

The complete reflection is summarized in table 3.1.

Table 3.1.: Increasing and decreasing Step 3 roles in the executable reconstruction.

Operation	Increasing	Decreasing
Step 3(a) boundary	predecessor-side A_i	successor-side B_i
Boundary insertion	after last sibling	before first sibling
Step 3(b) boundary	successor-side B_i	predecessor-side A_i
Acquired split side	left	right
Child extreme	rightmost child	leftmost child
Geometric anchor	right endpoint	left endpoint
Output insertion	after anchor	before anchor

3.8. Executable Anchor Clarifications

The point insertion in Step 3(c) is where the compact paper notation requires the most care. Two independent distinctions are necessary.

3.8.1. Curve Endpoint Versus Geometric Endpoint

The 1990 increasing step refers to the rightmost child $\{z_{m-1}, z_m\}$ and inserts the new point after z_m. A pair record, however, stores its endpoints in curve order, and its second curve endpoint is not necessarily its geometric right endpoint. Reading z_m mechanically as the second stored endpoint fails on valid reflected orientations.

For example, consider

$$(3, 2, 1, 4). \tag{3.10}$$

At $i = 4$, the acquired child is $P_2 = \{3, 2\}$. Inserting 4 after the second curve endpoint 2 would yield $(1, 2, 4, 3)$. Inserting after the child's geometric right endpoint 3 yields the correct order. The decreasing counterpart

$$(2, 3, 4, 1) \tag{3.11}$$

similarly requires insertion before the child's geometric left endpoint 2, not before its second curve endpoint 3.

The reconstruction therefore uses $r(C)$ for the increasing branch and $\ell(C)$ for the decreasing branch. This choice agrees with the earlier family-list account of Hoffmann et al. (1986), in which both endpoints occur in sorted family lists and the new point is inserted next to the located boundary item. It is nevertheless labeled an executable clarification rather than attributed to a fully explicit sentence in the 1990 presentation.

3.8.2. Odd-Index z_1 Output Anchor

Steps 1 and 2 compensate for the fact that z_1 has no lower-family pair. That boundary-pair correction does not cover every final output insertion. The 1986 description records a separate anomaly: on an odd iteration, if z_1 lies between the ordinary family-list boundary and the new point, the new point is inserted next to z_1 (Hoffmann et al. 1986, p. 175).

The executable increasing rule is

$$\text{if } i \text{ is odd and } a_i < z_1 < z_i, \quad a_i \leftarrow z_1, \tag{3.12}$$

where a_i is the geometric base anchor. The decreasing reflection is

$$\text{if } i \text{ is odd and } z_i < z_1 < a_i, \quad a_i \leftarrow z_1. \tag{3.13}$$

For the decreasing sequence prefix

$$(1, 2, 3, 4, 6, 7, 0), \tag{3.14}$$

the ordinary geometric base anchor at $i = 7$ is 2. Inserting 0 before 2 would leave 1 on the wrong side. Because $0 < z_1 = 1 < 2$, the output anchor is changed to z_1, and insertion before it gives the correct prefix order. The reflected sequence $(6, 5, 4, 3, 1, 0, 7)$ exercises equation (3.12). These cases demonstrate why boundary selection and output-anchor correction must be modeled as separate stages.

3.9. Ownership Transfer and Failure Semantics

A sibling split changes several linked facts at once: the old list ceases to exist, up to two nonempty output lists are created, the retained side stays with the old parent, the acquired side moves under N, and every moved pair must name its new list and parent. Conservation and unique ownership are required after a successful transition.

The reconstruction first forms a split plan without changing live state. Commit checks that the plan is current, that the proposed parent is live and cannot introduce a cycle, and that the partition is legal. It then updates the lists and ownership links and checks the resulting state. In the revised implementation, an update or postcondition error terminates sorting. Updates already performed are not rolled back, and the failed state must be discarded. No partially updated state is accepted as a completed iteration or returned as a sorting result.

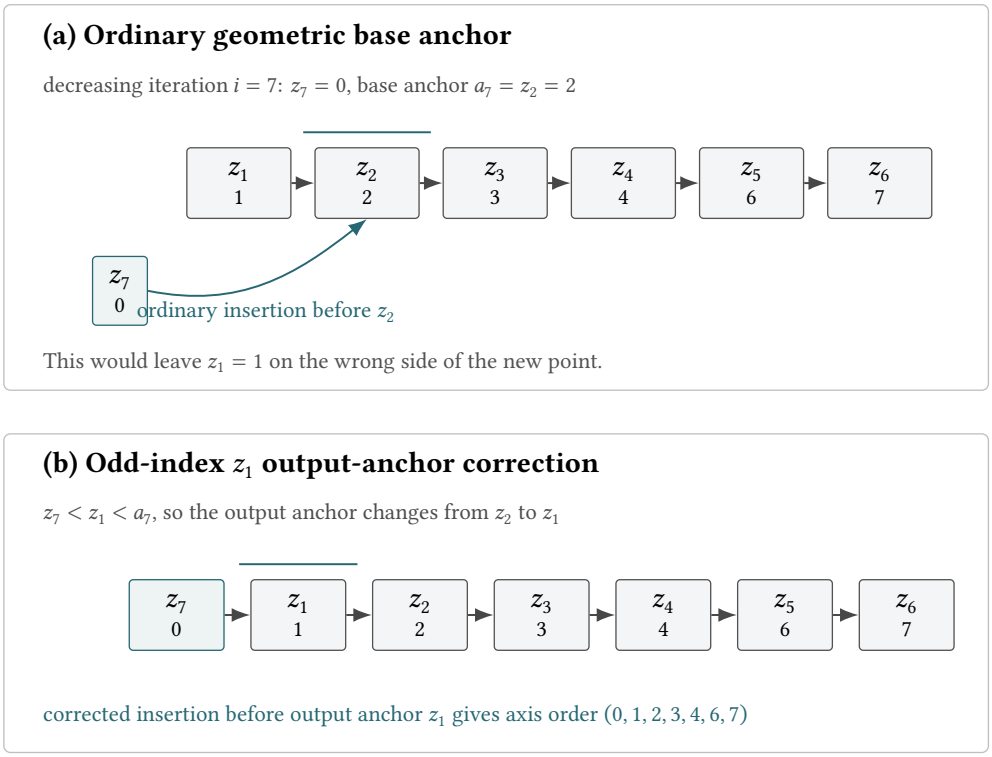


Figure 3.1.: Local output-anchor selection in decreasing Step 3(c) for the prefix $(1, 2, 3, 4, 6, 7, 0)$. The ordinary geometric base anchor is z_2. Because the odd-index condition $z_7 < z_1 < z_2$ holds, the output anchor is corrected to z_1 before z_7 is inserted.

The earlier implementation used rollback on these error paths. That recovery mechanism is distinct from the algorithmic requirement to preserve ownership on successful updates. The two versions and their experimental scope are distinguished in section 4.1; the concrete update is described in section 4.3.3.

3.10. End-to-End Procedure

The full valid-input sorting procedure is:

1. Materialize the input once and handle lengths zero, one, and two directly.
2. For $n \geq 3$, initialize S_3 by fixed-size ordering of the first three points, then create P_2, P_3, the two dummies, and their singleton sibling lists.
3. For each $i = 4, \ldots, n$:

   Step 1. Compute predecessor-side boundary A_i.

   Step 2. Compute successor-side boundary B_i.

   Orient. Choose increasing or decreasing Step 3 from the comparison of z_{i-1} and z_i.

   Update. Insert P_i, split and transfer any enclosed children, and insert z_i using the resulting geometric output anchor.

   Commit. Advance the processed prefix only after the output insertion succeeds.
4. Return the values obtained by traversing S_n.

At the end of each successful iteration, the required online conditions are:

- S_i contains each processed point exactly once and is strictly increasing;
- every processed finite pair exists in the family dictated by its parity;
- every finite pair belongs to exactly one live sibling list and names that list's owner as its parent;
- sibling order follows geometric left endpoints, parent chains are acyclic, and each chain reaches the appropriate dummy root; and
- split outputs are disjoint and preserve the input pair set.

The final return value is thus recovered from the maintained order S_n. Oracle certification may establish the valid-input precondition before this procedure is called, but oracle-sorted output is not an input to these steps. The implementation-level certification boundary is detailed in section 4.4.

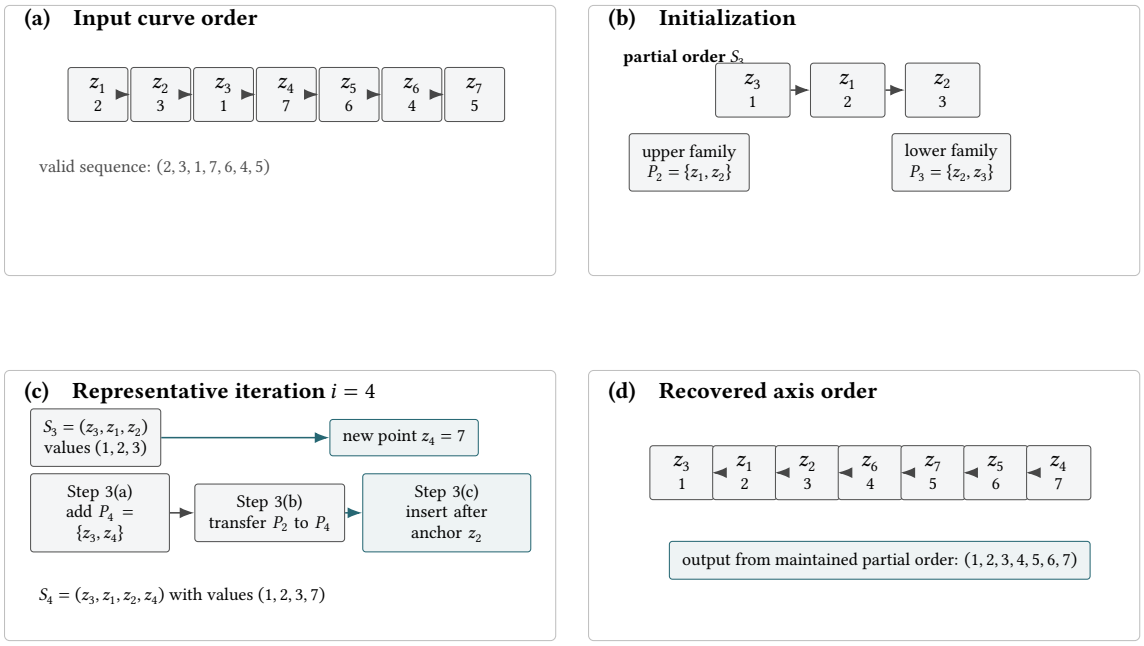


Figure 3.2.: Running example on the valid sequence $(2, 3, 1, 7, 6, 4, 5)$. The panels show the input, initialization, the representative update at $i = 4$, and the recovered axis order. At $i = 4$, P_4 acquires P_2 in Step 3(b), then Step 3(c) inserts z_4 after the geometric anchor z_2.

3.11. Correctness and Complexity Boundaries

The source-level correctness argument rests on the nesting property of same-family pairs and the locality lemma: the boundary pairs found beside z_{i-1} identify where P_i belongs, and newly enclosed children form a contiguous split side. The executable reconstruction turns those facts into the prefix and ownership invariants above. Focused counterexamples fix the ambiguous anchors; mirrored tests check the decreasing reconstruction; and full state diagnostics compare the maintained order and family forest with a deterministic replay. These are implementation correctness controls, not an independent proof that every unspecified detail has only one possible interpretation.

The theoretical papers and the thesis implementation have different backend claims. Hoffmann et al. (1986) use level-linked search trees, while Fung et al. (1990) reduce the required operations to singleton-list creation, boundary insertion, and splitting, then implement them with heterogeneous finger trees. The implementation preserves those abstract operations but realizes sibling lists as ordinary Python lists. A sibling split scans and materializes list contents, both insertion boundaries copy the current list, and ownership rebinding can depend on list size. Membership and ancestor checks can also scan lists in the minimal execution mode. The implementation is therefore an ordinary-list reconstruction of the paper-facing control flow, not an implementation of either theoretical linear-time data structure.

4. Implementation

4.1. Implementation Scope and Versions

The code separates an oracle-backed reference pipeline from the shared-core reconstruction of Fung et al. (1990). The reference function `simplified_jordan_sort` certifies the input, constructs family trees, statistics and a trace, and returns the oracle's sorted output. In contrast, `paper_jordan_sort_valid` executes the reconstructed algorithm and returns `state.partial_order.to_list()`. Its core does not use the oracle's output or a global rank map. It assumes a pre-certified valid Jordan sequence with distinct, mutually comparable values.

Both current backends propagate errors without rollback. Timings in chapter 6 belong to an earlier rollback-enabled version and measure neither the current programs nor a causal benefit of removing recovery work. Source bindings remain in section A.2.

Chapter 3 specifies initialization, both orientations of Step 1/2/3, and executable anchor clarifications. Here the focus is state representation and maintenance. The real tree shares the sorting loop and has bounded correctness evidence; cost and locality review remains open. Qualification does not establish a general linear-time theorem.

4.2. Data Representation

An immutable `PointRef(paper_index, value)` separates one-based paper identity from a point's comparable value. A `PairRecord` retains curve endpoint IDs and family membership; geometric endpoints are obtained by comparing their values. The distinction and its consequences for output anchors are specified in section 3.8.1.

`PaperJordanState` holds these records, the processed-prefix length, the partial order, the end-index-to-pair mapping, the sibling backend and two family roots. It also holds a fixed execution policy, optional trace and counters, and mandatory stage results. Sorting and diagnostics use the same runner and mirrored orientation branches.

4.2.1. Partial Sorted Order

`SortedOrderList` is doubly linked, with identity-based infinity sentinels and a point-ID-to-node mapping. Neighbor access and insertion use local links. Before mutation, insertion checks that the value lies strictly between its adjacent values. Final traversal excludes the sentinels and returns the original values. Mapping access uses Python dictionaries; its cost must not be silently treated as a deterministic worst-case constant.

4.2.2. Sibling Lists and Ownership

Each `SiblingList` stores ordered finite pair IDs and one owner. A pair caches its parent and containing-list IDs, together with its ordered child-list IDs. Every owned finite pair occurs in exactly one live list, whose owner equals its parent. A parent has at most two ordered child lists; finite parent chains must reach the same-family dummy without cycles. The family dummies are roots, not finite list items.

The ordinary backend retains this eager representation. Since commit `71e3e71`, the shared core uses boundary-list, owner and extreme-child access interfaces instead of reading those fields directly. General pair-liveness checks do not require the pair to be a list endpoint.

The tree backend stores stable pair handles in external leaves, with internal separator nodes, two endpoint fingers and a list header. Outer rib pointers point toward the root; interior pointers point toward children. Separators store the maximum left key. Rank is black height; span counts all leaves only in interior subtrees, the inward branch on a rib, and zero at the root. The header separately stores total size. Ownership belongs to the header and at most two parent child slots, avoiding per-leaf rebinding. Boundary slots only identify designated endpoints; they are not arbitrary-owner lookup caches.

4.3. Core–Backend Interface and Update Semantics

4.3.1. Initialization, Boundaries and Stage State

The initialization in section 3.4 materializes all point records but orders only the first three values; shorter inputs use fixed-size comparisons. Subsequent iterations use the predecessor and successor boundaries defined in section 3.5. The backend handles singleton creation, boundary insertion, splitting and ownership transfer; the partial-order structure handles point insertion.

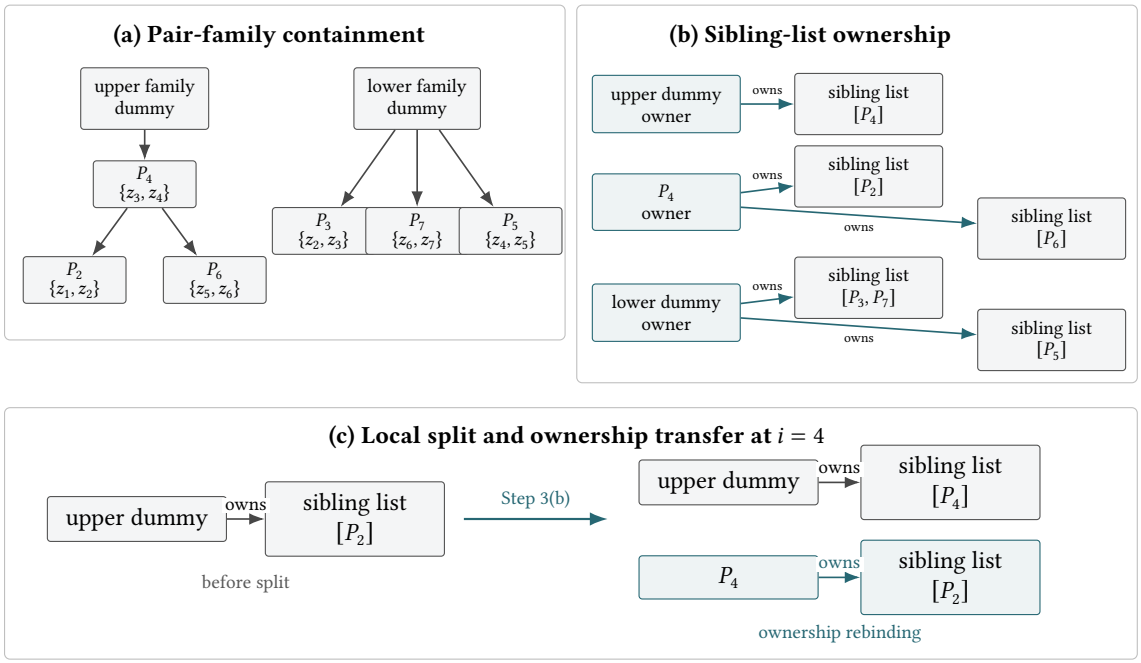


Figure 4.1.: Pair families and sibling-list ownership in the example of figure 3.2. Step 3(b) transfers P_2 from a list owned by the upper dummy to a child list of P_4.

Boundary selections and Step 3 results are stored even with trace disabled. Later stages consult them to reject incorrect order, repeated stages and inconsistent parameters. They are execution state, not merely optional logging. Optional backend-work observations are stored separately from these mandatory results; complete logical snapshots belong to the audit path.

4.3.2. Pair Insertion and Failure Propagation

For the insertion rules in sections 3.6.1 and 3.7, the core checks the boundary, registers the new pair and calls singleton creation or boundary insertion. A later error may leave that registration in place. The exception propagates; the state is discarded rather than restored or reused. This contract applies to both orientations.

4.3.3. Split Plans and Ownership Transfer

The ordinary backend implements the partition specified in sections 3.6.2 and 3.7 using an immutable `SplitPlan`. It records the original sequence and both ordered partitions before mutation. The plan sup-

ports stale-plan rejection, order checks and pair-set conservation; these functions remain necessary without rollback.

Commit checks the plan and ownership preconditions, prepares nonempty output lists and affected child slots, retires the input list, publishes the new lists, and rebinds the pairs. Touched-state postconditions are checked before success is returned. Rejection covers stale plans, straddling pairs, ownership mismatches, cycles, unowned parents and a third child list. Errors during mutation or postcondition checks propagate without restoring earlier state. Output-list preparation is ordinary update work, not a recovery backup.

The tree starts boundary insertion at a finger and repairs locally. A split with both sides nonempty races from both ends, detaches a component, splits its downward representation and joins by black height. Path sizes and header totals supply output sizes without enumerating leaves. Every successful split, including a one-sided result, retires its source ID and allocates fresh IDs for nonempty outputs; boundary insertion preserves its live ID. Pair handles, leaves and existing descendant groups survive transfer. Before Step 3(a), the core identifies the other group's transfer source without splitting it. Step 3(b) consumes that exact source; pending split/transfer state must clear before Step 3(c) inserts the output point.

4.3.4. Point Insertion and Completion

The backend exposes the relevant extreme child; the core selects its geometric endpoint and applies the separate odd-index z_1 anchor rule of section 3.8.2. These executable clarifications and their counterexamples remain in section 3.8; they are not attributed to fully explicit wording in the 1990 paper. The partial order performs the resulting adjacent insertion. Only after it succeeds does Step 3(c) advance `processed_count`.

4.4. Certification, Diagnostics and Replay

The wrapper `certified_paper_jordan_sort` runs the oracle and rejects invalid candidates before entering the pure sorter. Certification is outside the minimal core's timing scope. An internal exception is propagated by both sorting and diagnostic entry points: neither retries, returns a partial output, nor falls back to the oracle or reference pipeline. Output recovery is reached only after the runner succeeds.

`paper_jordan_diagnostics_valid` selects `checked` and audits initialization and every completed prefix. It checks point identity and values, partial-order links and order, exact pair membership, ownership

and parent chains, stage records, trace and metrics. Deterministic replay reconstructs a fresh state using the same core, backend type and immutable configuration. It compares backend identity, partial order, registries, stage results, trace, work observations, metrics, and logical and physical snapshots. Unavailable backends fail explicitly rather than reverting to ordinary lists. This is same-core state-consistency evidence, not a second independent algorithm or a replacement for the external sorted-output check.

Qualification v2 instead advances two independently initialized checked instances through the same pausable core, audits and compares every completed prefix, then performs one final fresh checked replay. A separate fresh minimal call checks the measured configuration. It avoids reconstructing every prefix from scratch; this changes replay coverage and is explicitly versioned. Snapshots reuse one audit within a validation call, never a cross-call "already checked" cache. Old stage and trace records remain checked.

Failure-injection tests separately check propagation from pair insertion and split postconditions, including checks retained in `minimal`. Public-call tests require one split attempt and no output recovery after failure. They impose no rollback or failed-state reuse requirement.

4.5. Execution Policies and Observation Costs

Five immutable policies control trace recording, selected operation counters and complete backend validation after split commits. `checked` enables all three; `minimal` disables them. Intermediate policies isolate observation costs. All use the same Step 1/2/3 sequence and retain stage results, necessary local checks, input materialization and output recovery. The revised implementation has no rollback under any policy.

The tree's detailed work ledger is a separate configuration switch: minimal alone does not disable it. Qualification and calibration explicitly select `detailed_work=False`, suppressing field accumulation, ledger snapshots and per-operation reports. Local geometry and pending checks remain; access wrappers and switch checks still have runtime cost.

The archived studies instead timed the rollback-enabled implementation, including recovery-state preparation in `minimal`; a successful call need not actually roll back to incur that preparation. The historical scope is specified in section 5.3.3.

For each successful split, the ordinary backend reports an increment equal to the input-list length for both `split_items_scanned` and `split_items_copied`. They are logical split-volume measures, not exact interpreter-level access counts. They omit boundary-insertion copies, membership scans and repeated validation/materialization passes. Retained historical counter names do not establish

that splits dominate runtime. Logical volumes, directly observed work and wall-clock timings are therefore distinct forms of evidence. The shared core accumulates backend-reported work under an identified metric schema; it does not infer tree copying from the input-list length.

4.6. Actual Costs and Complexity Boundary

In the ordinary backend, either boundary insertion constructs a new Python list containing the existing IDs and the new pair. Rear insertion is not an in-place `list.append()`. Both orientations copy work proportional to the current sibling-list length. Split additionally scans and partitions the list, materializes its outputs and rebinds pair ownership.

"Local" describes the objects a safety check concerns, not constant cost. The ordinary backend's liveness check uses `pair_ids.count(pair_id)` for membership, and backend parent checks scan the containing list at each visited ancestor. For an ancestor chain A, this contributes $O(\sum_{a \in A} |L(a)|)$, where $L(a)$ contains a, rather than a constant per ancestor. These costs remain in `minimal`; complete checked validation and prefix replay add separate untimed work.

The valid increasing input $(1, 2, \ldots, 2m)$, $m \geq 2$, exposes a quadratic copying lower bound even without split. Starting from one pair per family, boundary insertions grow the sibling lists to m and $m - 1$ while Step 3(b) skips splitting. The number of existing pair IDs copied is

$$\sum_{j=1}^{m-1} j + \sum_{j=1}^{m-2} j = (m-1)^2. \tag{4.1}$$

Thus this input family incurs $\Omega(n^2)$ copying, including in `minimal`. This is a source-derived bound on one component of work, not a curve fit or a tight total bound for all inputs.

The ordinary backend instantiates neither the level-linked structure of Hoffmann et al. (1986) nor the heterogeneous finger-search structure of Fung et al. (1990). For the implemented tree, the source-linked draft analyzes rank changes across successive joins, root-side rib walks and one potential for mixed insertion/split sequences. Its target is amortized $O(1)$ boundary insertion and $O(\log(s + 2))$ split, with s the smaller output size. A second list-size potential supplies a conditional linear total. These conclusions depend on representation preservation, legal local calls, unit-cost comparison and expected/amortized dictionary operations. The full review remains open; tests, calibration and the absence of leaf scans do not prove these bounds or unconditional Python running times.

5. Experimental Method

5.1. Study Scope and Version Boundary

Experimental algorithmics studies concrete programs, inputs and machines (McGeoch 2012); algorithm engineering separates implementation, analysis and evaluation (Sanders 2009). Accordingly, the current matched protocol and the historical measurements are distinct studies. The current ordinary-list and tree implementations have input qualification and actual batch calibration, but no formal paired samples. Quantitative results in chapter 6 still describe the earlier rollback-enabled ordinary-list implementation. Neither calibration nor a later checker changes an old execution's source identity.

5.2. Current Inputs and Qualification

The frozen common set contains 140 unique rank inputs at ten lengths: 32, 33, 64, 65, 128, 129, 256, 257 and 512, 513, with two strict-flat and four each of low, medium and heavy nesting per length. Fixed constructors, seeds, candidate budgets, rank deduplication and first-eligible retention preceded timing. These are structural constructions, not random independent observations. All 140 cases pass both backends' v2 checks. Their 3,788 recorded splits per backend all have an empty output side: they exercise ownership transfer but not the tree's interior two-sided split path.

A separate targeted set uses the existing construction at $n = 128, 129$, $k = 3, 7, 19$, with each direction and its mirror: twelve cases, all qualified. Actual traces contain twelve two-sided splits per backend, with smaller sides 3, 7 and 19 occurring four times each. Each source has size 61 and a finite owner; later direct updates affect only the retained side. This bounded coverage does not represent all scales, ratios or follow-up operations. The two sets remain separate, not 152 independent random samples.

For each input/backend, v2 advances two independent checked instances, compares every completed prefix, and executes a third fresh checked replay at the end. Another fresh minimal call uses the intended timing configuration. Each output is compared to external sorting, without feeding that expectation into the core. These checks establish finite execution evidence, not a general proof.

The old fresh-replay-at-every-prefix protocol remains a bounded reference and is not described as identical coverage.

5.2.1. Historical Input Protocol

The archived formal study instead used 60 cases: at each $n \in \{32, 64, 128, 256, 512\}$, one flat, one nested and ten seeded incremental cases. Generation seed `20261201` and case identities determine the inputs. Duplicate hashes were rejected within family–size groups, not by a global rank rule. All 60 passed oracle validity, distinctness, structural profiling and checked ordinary-list diagnostics before warm-up. Failures stopped the run; checked outputs were compared externally. The ten-to-one family imbalance remains visible in historical aggregate results.

5.3. Compared Implementations and Call Scopes

The current comparison uses the same public Jordan core with ordinary-list or heterogeneous-finger storage, both minimal; the tree explicitly sets `detailed_work=False`. Every call initializes fresh state and recovers its output. Production source hashes match the qualified versions. Source or configuration changes require checking which qualification and measurements must be renewed. A tool-only archive check is recorded separately.

5.3.1. Python Sort

In the historical three-way comparison, `python_sort` sorts directly, without Jordan recognition or structural analysis.

5.3.2. Reference Pipeline

`simplified_jordan_reference` includes oracle validation, state and family construction, statistics, trace and result assembly. It is a pipeline, not a sorting-only primitive.

5.3.3. Historical Ordinary-List Core

`simplified_jordan_paper_ordinary_list` times the pre-certified rollback-enabled minimal core, including local checks, mandatory stage state, recovery preparation and output recovery. Successful calls still pay recovery preparation costs. Its ratios to the reference retain this difference in scope; they do not isolate the benefit of rollback removal or of tree storage.

5.4. Timing, Calibration and Scheduling

Current timing uses `time.perf_counter_ns()`. The interval includes fresh initialization, the main loop, output recovery, the batch loop and output append. External comparisons, pre-batch garbage collection, compression and persistence are outside it. GC remains enabled, with thresholds recorded; necessary local checks are not removed. A batch is the statistical unit: dividing its duration by B does not create B independent observations.

Calibration tries $B = 1, 2, 4, 8, 16, 32, 64, 128$ in order, three batches per backend at each attempted value. It chooses the first common B for which both median batch durations reach 10 ms, irrespective of their ratio. All attempts and outputs are retained. At source `c5b5ac3`, the common set selected B=1,2,4,8,16,32 for respectively 24,20,20,28,21,27 inputs; all twelve targeted inputs selected B=8. Independent reconstruction accepted both campaigns, within their separate 600-second budgets. These are granularity decisions, not formal speed comparisons or statistical-precision guarantees.

5.4.1. Scheduling and Noise

Calibration alternates the first backend by case index, candidate index and repeat. The unexecuted formal protocol shuffles cases with seed `20261015`, uses five warm-up calls per backend and twenty paired batches at each frozen B, alternating first position by case index and round. Each backend is first ten times. Failures stop subsequent cases; timeouts preserve committed batches and leave explicit unrun statuses. No outlier removal or speed-based input replacement is allowed.

Historically, each of three algorithms instead received five warm-ups and twenty individually timed calls per input. Case/order seeds were `20261203/20261202`, with cyclic position rotation. The old wrapper disabled GC only around the measured call and restored its prior state; that policy is not inherited by the current comparison. Controlled scheduling reduces fixed-position bias, not all thermal or operating-system noise (Mytkowicz et al. 2009). Repetition alone does not establish adequate statistical power (Kalibera and Jones 2013).

5.4.2. Recorded Environment

Each current archive binds source hashes, Python executable/version/flags, platform, CPU, RAM, power observations, GC thresholds and monotonic-clock metadata. Calibration and formal execution must match source and recorded machine/runtime settings. Tests, qualification and compilation were not run concurrently with calibration. The historical formal archive records Apple M4/arm64, 16 GB RAM, ten logical CPUs, macOS 26.6 and CPython 3.12.4, with its own preflight; these are not claims about today's runtime.

5.5. Aggregation and Statistical Units

The current formal protocol, once executed, reports twenty batch-derived per-call durations per input/backend, their median and R7-interpolated quartiles and IQR, and the paired ratio of backend medians. Related constructions and mirrors remain dependent; cross-input summaries are descriptive.

The historical archive contains $60 \times 3 \times 20 = 3600$ measured calls and 180 case–algorithm summaries. Its quartiles use median-of-halves, not R7. The 45 family–size–algorithm groups aggregate case medians; ratios are formed within inputs before equal-case aggregation. Counter associations are exploratory, not causal decompositions of runtime.

5.6. Evidence Binding and Acceptance

The historical formal/pilot sources are `98868b1`/`01f6480`. The current 140-case and twelve-case qualification producers are `3cafb44` and `668371f`; checker/calibration source is `c5b5ac3`. Full hashes and immutable archive locations are indexed in section A.2. New checking does not relabel old runs.

Each current record is independently compressed, synchronized and atomically committed. A terminated worker cannot erase earlier completed records. A new archive stores its configuration before execution and binds records, decisions and environment by manifest. Resource wall time includes checking and storage and is separate from a measured call. Old archives are never resealed to imply new execution.

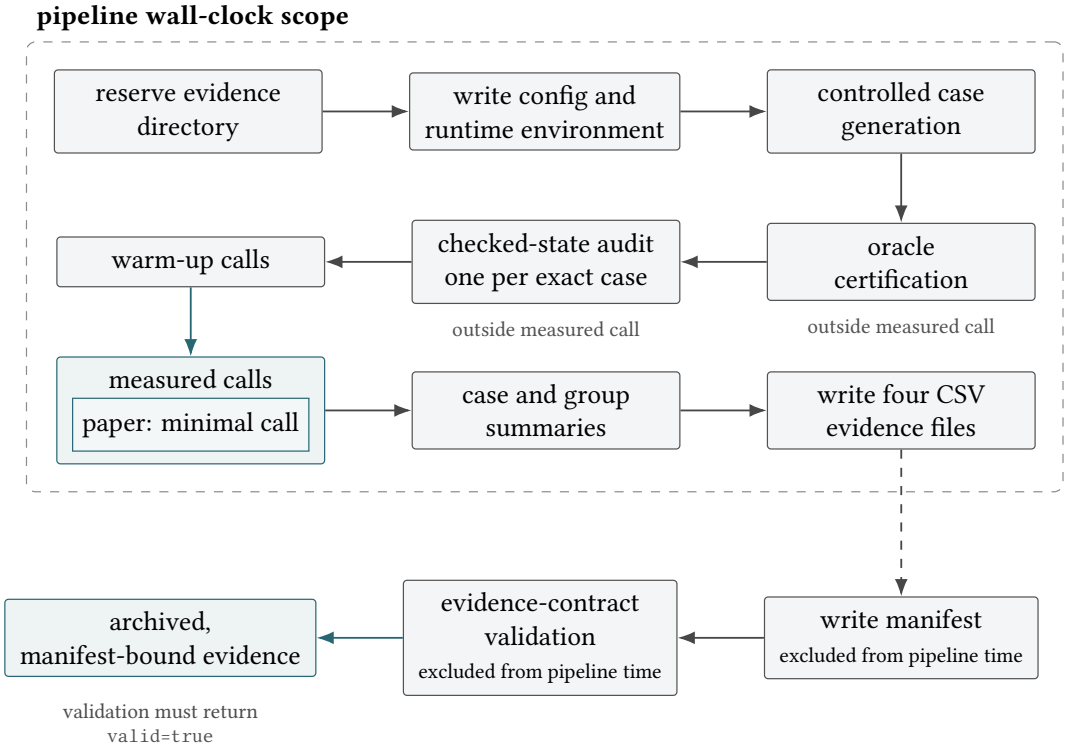


Figure 5.1.: Historical rollback-enabled formal pipeline. Current qualification and batch calibration have separate identities and do not supply new formal results.

5.6.1. Fail-Closed Validation

The current archive checker verifies schemas, source blobs, input identities, outputs, event order, sizes, owners, counters and list-ID lifetimes. A live-ID map checks current membership; a separate seen-ID set prevents a singleton or split output from reusing any retired ID. Boundary insertion keeps its live ID. Read-only rechecks accept both existing datasets without rerunning sorting. The measurement validator independently reconstructs candidate order, every attempt, the first qualifying common B and output correctness. A process exit code alone is insufficient. Historical validation additionally reran the old checked diagnostics; that behavior is not attributed to the new read-only check.

5.7. Interpretation Boundary

Qualification, calibration, proof review and formal measurement are separate gates. Formal samples remain unexecuted, and the cost/representation/locality review remains open. No calibration ratio enters the results chapter, no historical pilot is renamed as current evidence, and no speed outcome proves the target asymptotic bound.

6. Results

6.1. Evidence Status and Correctness

The primary results use the archived, manifest-bound formal evidence and analysis artifacts regenerated from that archive. The directional-replication comparison in section 6.7 additionally uses archived, manifest-bound pilot evidence. Exact evidence locators and version bindings are given in section 5.6 and table A.2. The recorded validation reports accepted the formal archive without errors and confirmed 3600 raw rows, 180 case summaries, 45 group summaries, and 60 checked-state audits.

All performance results in this chapter concern the earlier rollback-enabled ordinary-list implementation described in section 5.3.3. They are retained as historical baseline evidence. All captions labelled "Historical baseline refer to this version and the timed scopes in section 5.4; they do not describe the current no-rollback core. The revised implementation in chapter 4 has not been measured by this archived protocol; none of the reported timings or ratios quantify the effect of removing rollback.

Across the archived evidence, all 3600 measured rows belong to 60 exact cases that each passed one untimed checked-state audit before timing. Every row has an oracle-certified input, correct sorted output, and no recorded error. All 180 case summaries report correct, error-free cells, and all 60 checked-state audits pass. The row-level audit field carries the corresponding case-level outcome; it does not represent 3600 distinct audit executions. This establishes empirical correctness for the pre-specified cases; it is not a proof for every valid Jordan sequence.

6.2. Runtime by Input Size

All absolute times in this section belong to the single recorded execution described in section 5.4.2: Apple M4/arm64 with 16 GB memory, macOS 26.6, and CPython 3.12.4. The reference value times the complete oracle-backed reference pipeline. The paper value times only the pre-certified ordinary-list core in `minimal` mode; oracle certification and the checked diagnostic are outside timing. The Python value includes its own list construction inside the algorithm call.

The median exact-case runtimes are shown in table 6.1 and figure 6.1.

Table 6.1.: Historical baseline. Median exact-case runtime by input size. Values are milliseconds; each size contains twelve exact cases.

n	Cases	Python	Reference	Paper
32	12	0.000 791	0.181 177	0.586 906
64	12	0.001 417	0.585 667	1.282 198
128	12	0.002 635	2.077 959	2.815 083
256	12	0.005 208	7.929 646	6.721 250
512	12	0.010 584	33.014 458	18.716 687

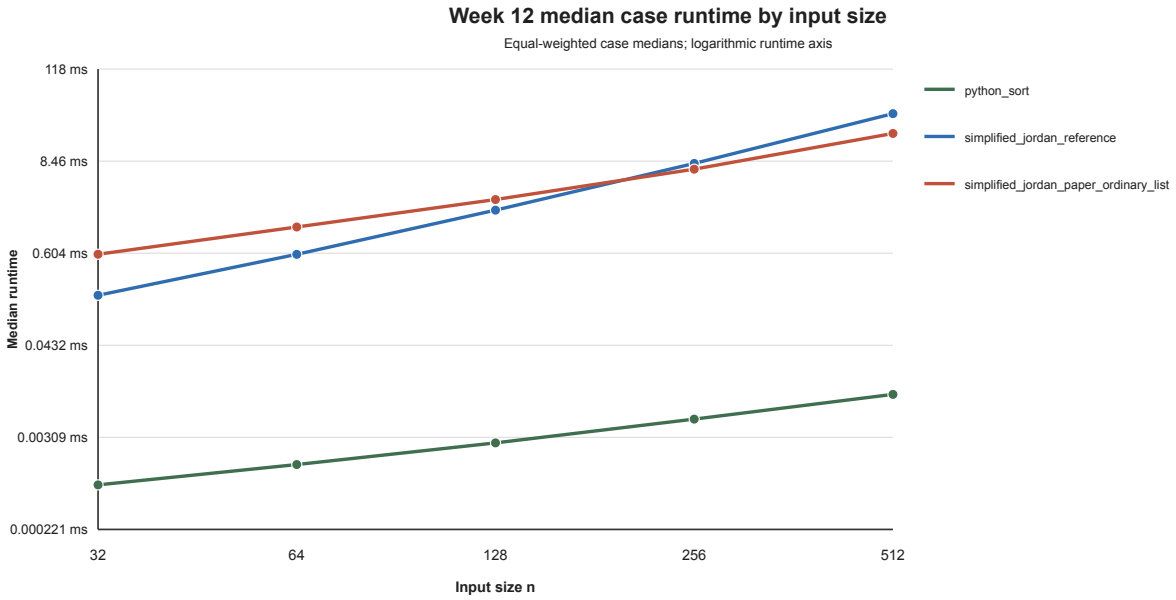


Figure 6.1.: Historical baseline. Median runtime by input size in the formal experiment. The logarithmic vertical axis displays equal-weighted exact-case medians under the distinct timed scopes defined in section 5.3.

At every tested size, Python sort has the smallest median call time. Under the fixed pipeline scopes, the paper call is larger than the reference call at $n = 32$, 64, and 128, and smaller at $n = 256$ and 512. This is a comparison of the timed calls shown above, not an end-to-end or asymptotic performance claim.

6.3. Matched-Case Runtime Ratios

The primary comparison is the paper/reference ratio computed within each exact case and then aggregated across the twelve cases at a size. A ratio above one means that the pre-certified paper call has the larger median time; a ratio below one means that it has the smaller median time. Because the reference and paper timed scopes differ, these values are pipeline-scope ratios rather than like-for-like end-to-end speedups.

Table 6.2.: Historical baseline. Median exact-case paper/reference ratio by input size.

n	Cases	Median ratio
32	12	3.226 420
64	12	2.202 394
128	12	1.351 064
256	12	0.850 597
512	12	0.567 187

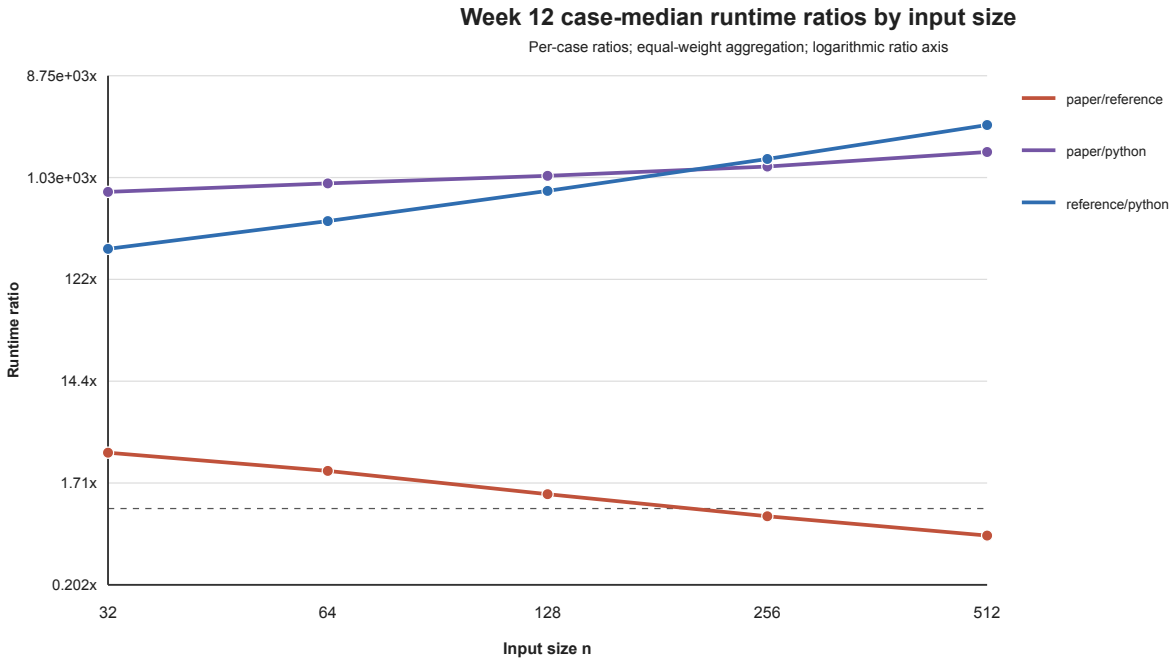


Figure 6.2.: Historical baseline. Case-median runtime ratios by input size in the formal experiment. Ratios are formed within exact cases before equal-weight aggregation; the dashed line marks a ratio of one.

The median exact-case paper/reference ratio declines across all five tested sizes, from 3.226 at $n = 32$ to 0.567 at $n = 512$. It remains above one through $n = 128$ and is below one at $n = 256$ and $n = 512$. Thus, the observed crossover lies between the tested sizes 128 and 256. The five points describe the pre-specified sample and do not establish an asymptotic rate.

Across all sixty exact cases, the overall median ratios are given in table 6.3.

Table 6.3.: Historical baseline. Overall median exact-case runtime ratios across all sixty cases.

Comparison	Cases	Median ratio
Paper/reference	60	1.351 064
Paper/Python	60	1117.902 852
Reference/Python	60	784.015 810

The two research implementations are therefore orders of magnitude above the Python baseline in the overall exact-case ratios. These medians aggregate the tested cases across sizes and are not asymptotic growth estimates.

6.4. Family-Specific Ratios

The paper/reference ratio also decreases with size within each of the three controlled families, as shown in table 6.4.

Table 6.4.: Historical baseline. Median exact-case paper/reference ratio by family and input size.

Family	Cases/n	32	64	128	256	512
Flat	1	2.550 795	1.609 025	0.919 051	0.509 697	0.281 381
Nested	1	2.563 670	1.748 004	1.099 397	0.726 798	0.525 490
Incremental	10	3.281 790	2.217 258	1.358 474	0.859 922	0.569 733

The flat case first falls below one at $n = 128$; the nested and incremental families first fall below one at $n = 256$. These crossing statements apply only to the listed constructions and sizes. Flat and nested each contribute one deterministic case per size, whereas incremental contributes ten seeded cases.

The overall size medians are consequently not balanced family averages. Ten of the twelve cases at each size are incremental, so the incremental rows have the largest influence on the size-level median. The family table is retained to make that composition visible.

6.5. Runtime Variability

Relative IQR is the IQR divided by the median for one exact case-algorithm cell. The analysis flagged cells at or above 0.25 for inspection without removing them.

Table 6.5.: Historical baseline. Relative-IQR summary over exact case-algorithm cells.

Algorithm	Median	Maximum	Flagged cells
Python sort	0.0556	0.2520	1
Reference pipeline	0.0119	0.0661	0
Paper ordinary-list	0.0152	0.0242	0

The only flagged cell is `incremental_valid_n512_001` under Python sort, with relative IQR 0.251996. It remains in every aggregation. No reference or paper cell reaches the inspection threshold. The threshold is a study-specific flag, not a universal outlier definition, and the table does not assign a cause to the observed variation.

6.6. Measured Calls and Pipeline Wall-Clock

The captured measured calls sum to the values in table 6.6.

Table 6.6.: Historical baseline. Captured measured-call time by algorithm.

Algorithm	Measured calls	Total time (s)
Python sort	1200	0.004 750 632
Reference pipeline	1200	10.445 239 255
Paper ordinary-list	1200	6.919 634 017
All algorithms	3600	17.369 623 904

These sums contain only measured algorithm calls. They exclude warm-ups, generation, certification, diagnostics, summaries, and file operations and must not be interpreted as complete experiment duration.

The recorded formal pipeline wall-clock is 837.682 385 541 s. Its scope begins at formal evidence-directory reservation and covers configuration and environment writes, case generation, oracle certification, checked diagnostics, warm-ups, measured calls, summary construction, and CSV writes.

Manifest writing and validation are excluded. This is a pipeline duration, not algorithm runtime, and it is not allocated among the three algorithms.

6.7. Pilot–Formal Directional Replication

The pilot and formal experiments are compared only through ratios formed within each run. Their absolute runtimes are not pooled. The paper/reference size medians are shown in table 6.7.

Table 6.7.: Historical baseline. Paper/reference size medians in the pilot and formal experiments.

n	Pilot	Formal
32	3.095 548	3.226 420
64	2.148 361	2.202 394
128	1.354 239	1.351 064
256	0.817 629	0.850 597
512	0.564 000	0.567 187

For paper/reference, paper/Python, and reference/Python, the size-rank Spearman coefficient between the pilot and formal experiments is 1.0. Each comparison has matching direction on all four adjacent-size transitions and remains on the same side of ratio one at all five sizes.

The formal experiment thus reproduces the directional trend observed in the pilot experiment. This does not show equality between runs, provide a pooled estimate, or establish hardware-independent absolute time.

6.8. Exploratory Relationships

The analyses in this section are explicitly exploratory. They use the twelve mixed-family cases at each fixed size and retain the unbalanced family composition. The coefficients are descriptive and do not support causal or asymptotic interpretation.

6.8.1. Structure and Runtime

Table 6.8.: Historical baseline. Within-size Spearman coefficients between runtime and two structural measures.

n	Ref./depth	Ref./density	Paper/depth	Paper/density
32	0.814	0.895	−0.246	−0.238
64	0.885	0.944	0.011	0.112
128	0.856	0.944	0.414	0.469
256	0.897	0.979	0.448	0.559
512	0.932	0.951	0.340	0.462

Reference runtime has a positive descriptive association with both maximum depth and containment-pair density at every size. The corresponding paper coefficients are smaller and vary in sign or magnitude across sizes. This does not show that paper runtime is independent of structure; it records only the observed coefficients in this generated sample.

6.8.2. Checked Counters and Paper Runtime

Checked diagnostic counters and paper runtime come from different policies: the counters are collected by an untimed `checked` diagnostic, while runtime uses the `minimal` paper call. Their within-size Spearman coefficients are shown in table 6.9. The archived "Scans" and "Copied" fields measure logical split input volume; they do not count every element access or copy. Boundary-insertion copies and membership or ancestry scans are outside those fields. Their definitions and limits are explained in section 4.5.

Table 6.9.: Historical within-size Spearman coefficients between logical checked counters and rollback-enabled minimal paper runtime (source `98868b1`).

n	Scan volume	Splits	Copy volume	Transferred
32	0.919	0.965	0.919	0.919
64	0.646	0.842	0.646	0.598
128	0.418	0.429	0.418	0.260
256	0.326	0.319	0.326	0.249
512	0.417	0.253	0.417	0.285

All four selected counters have positive descriptive associations with minimal paper runtime at every tested size, although magnitudes differ by size and counter. The relationship is not a causal cost decomposition: the checked diagnostic work is observed outside the timed minimal call.

The `paper_invariant_checks` counter is constant across the twelve cases within each size: 30, 62, 126, 254, and 510 checks from $n = 32$ through 512. Its within-size Spearman coefficient is therefore undefined and remains empty in the analysis artifact; it must not be interpreted as zero association.

6.9. Results Boundary

The formal evidence supports empirical correctness on the sixty pre-specified cases, the reported runtime distributions and matched-case ratios, and directional replication of the three within-run ratio series. It also supports the labeled exploratory coefficients above. It does not establish linear-time Jordan sorting, asymptotic complexity from five sizes, recognition performance, causal structural effects, representative sampling of all Jordan sequences, or a like-for-like end-to-end paper/reference speedup. It also does not establish the runtime of the revised core or any heterogeneous finger-tree backend.

7. Limitations

7.1. Scope of the Limitations

The implementation and formal experiment establish a reproducible ordinary-list reconstruction of the paper-facing sorting control flow and its behavior on the pre-specified valid inputs. They do not establish every property of the theoretical 1990 algorithm or every behavior of Jordan-sequence inputs. The following boundaries determine how the implementation and empirical results may be interpreted.

7.2. Implementation Version and Failure Contract

The archived performance results describe the earlier rollback-enabled implementation. The revised core removes recovery backups and propagates internal update errors without restoring state. The caller must discard failed state; successful-update invariants do not provide an exception-recovery guarantee. This contract applies to both `checked` and `minimal` execution.

Removing recovery work leaves ordinary-list construction, copying, scanning, ownership rebinding, and local postcondition checks in place. The archived measurements cannot determine the revision's runtime or the size of any performance change. Those questions require a version-bound experiment; a causal claim about rollback removal additionally requires an appropriate matched comparison. Unchanged logical counters do not supply that evidence.

7.3. Paper Reconstruction and Backend Fidelity

The implemented paper core follows the reconstructed Step 1/2/3 control flow, maintains a partial sorted order and sibling-list ownership state, and recovers its final output from that maintained order. Some executable details, including the geometric endpoint used by Step 3(c) and the odd-index z_1 output-anchor adjustment, resolve omissions in the short 1990 description through counterexamples,

the related 1986 account, mirrored cases, and bounded exhaustive validation. This supports a paper-facing reconstruction, but not a claim that the implementation is the unique verbatim interpretation of every omitted detail.

More importantly, the historically measured backend uses ordinary Python lists. Sibling-list scans, copies at both insertion boundaries, slicing, list materialization, and ownership rebinding require work that can depend on the represented list length. Membership and ancestor-chain scans also remain in `minimal`. Indeed, equation (4.1) establishes quadratic copying work on strictly increasing inputs with no splits. This source-level lower bound does not supply a tight overall runtime bound. That measured implementation does not instantiate level-linked search trees, heterogeneous finger trees, or the theoretical dynamic split/update backend. Consequently, its observed runtime cannot be presented as an implementation of the paper's linear-time data structures or as evidence that the ordinary-list program runs in linear time. The later real tree has separate bounded qualification and calibration; its general proof review and formal matched performance evidence remain open.

7.4. Tested Sizes and Asymptotic Interpretation

The formal protocol contains five input sizes: 32, 64, 128, 256, and 512. The decreasing paper/reference ratio and the observed crossing between the tested sizes 128 and 256 describe this finite set of measurements. Five size points do not identify a Big-O class, prove a crossover outside the tested constructions, or establish the asymptotic behavior of either pipeline. The result must therefore remain an observed trend over the tested sizes rather than an asymptotic-complexity conclusion.

7.5. Non-Equivalent Timing Scopes

The paper/reference ratio does not compare equivalent end-to-end pipelines. `simplified_jordan_reference` is timed as its complete oracle-backed reference pipeline, including oracle validation, family-tree construction, structural profiling, and reference output work. The paper measurement times only the pre-certified ordinary-list core in `minimal` mode. Oracle certification and the full checked diagnostic occur before paper timing.

The ratio is therefore useful as a reproducible comparison of the two fixed timed scopes, but it is not a like-for-like end-to-end speedup. In particular, the ratios below one at $n = 256$ and $n = 512$ mean only that the paper timed call has the lower exact-case median under these scopes. They do not establish general superiority of the paper pipeline or its theoretical algorithm.

7.6. Output Provenance and Recognition

The two sorting paths have different output provenance. The reference pipeline returns an oracle-derived sorted list. In contrast, `paper_jordan_sort_valid` does not read `oracle_result["sorted"]`; for inputs of length at least three, it returns values traversed from `state.partial_order.to_list()`. Oracle-derived-output statements must therefore remain restricted to the reference path.

The paper core also assumes a pre-certified valid input. The public `certified_paper_jordan_sort` wrapper invokes the oracle to enforce that precondition before calling the core, but certification is outside the timed paper call and does not make the core a recognition algorithm. The formal experiment contains only oracle-certified valid-input sorting cases and explicitly records `recognition_separate=true`. It cannot support claims about invalid-input recognition accuracy, failure reasons, or recognition runtime.

7.7. Case Construction and Generality

The formal experiment contains sixty controlled cases from three generators. At each size, the sample contains one flat case, one nested case, and ten seeded incremental cases. It is therefore intentionally unbalanced, and the ten incremental cases have the greatest influence on each overall size median.

These constructions expose selected flat, nested, and incremental patterns; they are not a representative sample of all Jordan-sequence distributions. The all-correct result is empirical evidence for the pre-specified cases, not a proof for every valid Jordan sequence. Generalization to other valid constructions, other family balances, or larger sizes is not established by this experiment.

7.8. Execution Environment and Cross-Experiment Replication

The absolute runtimes from the formal experiment come from one execution in the recorded Apple M4 environment running CPython 3.12.4. Warm-ups, repeated calls, randomized scheduling, and environment checks reduce selected sources of measurement noise, but one run on one machine does not establish hardware- or software-independent absolute time.

The pilot and formal experiments were executed in different recorded environments; their absolute runtimes are therefore not pooled. The replication analysis forms ratios within each run and compares size ordering, adjacent-size direction, and position relative to ratio one. Matching directions support reproducibility of the observed ratio pattern, but they do not prove equality between runs or provide a pooled performance estimate.

7.9. Exploratory Structure and Counter Relationships

The reported Spearman coefficients use twelve mixed-family cases within each size. Their family composition is unbalanced, and the sample is too small and controlled to identify causal structural effects. The coefficients describe associations in the generated cases only; they do not show that depth, containment, or family membership causes a runtime change.

The operation counters introduce an additional scope difference. They are collected by an untimed `checked` diagnostic, whereas paper runtime comes from the `minimal` call. Positive counter/runtime associations are therefore not a causal cost decomposition of the timed implementation. The undefined within-size coefficient for constant `paper_invariant_checks` must likewise remain undefined rather than being reported as zero.

7.10. Runtime Variability and Statistical Reach

Each exact case-algorithm cell contains twenty measured calls, summarized by its median and median-of-halves IQR. These repeated calls characterize timing variation for a fixed generated case; they are not twenty independent input samples. Median and IQR reduce the influence of isolated fluctuations but do not remove persistent system effects. The single cell at or above the study's relative-IQR inspection threshold remains included, and the evidence does not identify its cause.

The overall ratios are medians across exact-case ratios. They summarize the pre-specified case set and its unbalanced family composition, not a population parameter for all valid Jordan sequences. Absolute runtime and ratio precision should consequently be reported only to the level needed to reproduce the archived tables, without implying broader statistical certainty.

7.11. Consequences for the Thesis Conclusions

Within these limits, the thesis may conclude that the ordinary-list paper core independently recovers sorted output for the pre-specified oracle-certified cases; that the evidence and diagnostics for those cases validate successfully; and that the paper/reference pipeline-scope ratio declines across the five tested sizes with the same directional pattern in the pilot and formal experiments. It may also report the explicitly exploratory associations preserved in the analysis.

The thesis may not convert those observations into claims of linear time, asymptotic complexity, theoretical-backend performance, end-to-end speedup, recognition behavior, causal structural effects, representative sampling, or hardware-independent absolute runtime.

8. Conclusion

8.1. Summary

This thesis translated the control flow of the 1990 Simplified Jordan Sorting algorithm into an executable and auditable Python implementation. The reconstruction maintains a partial sorted order, upper and lower pair families, and sibling-list ownership while executing the paper-facing Step 1/2/3 procedure. It makes explicit the orientation, endpoint, and first-point cases needed by the implementation and records where those choices are reconstruction decisions rather than uniquely determined statements of the paper.

The concrete backend uses ordinary Python lists. It therefore separates the incremental algorithmic structure from the specialized data structures on which the historical linear-time results depend. The thesis further separates the independently recovering paper core from an oracle-backed reference pipeline: the paper core obtains its output from its maintained partial order, whereas the reference path deliberately returns oracle-derived sorted output. Fixed execution policies, checked diagnostics, deterministic replay, and explicit sibling-list ownership checks make this boundary observable and testable. The revised implementation propagates internal update errors and requires the caller to discard failed state; it no longer restores updates through rollback.

The archived empirical study evaluated the earlier rollback-enabled implementation on oracle-certified valid inputs under a pre-specified protocol. It compared Python sort, the complete oracle-backed reference pipeline, and the pre-certified paper core in `minimal` mode. Archived, manifest-bound evidence and a separate evidence-contract validator preserved the cases, schedules, correctness fields, summaries, and audit records used for the analysis.

These measurements remain evidence about the earlier version. They do not measure the revised core or establish a speed improvement from removing rollback. The implementation account and the runtime conclusions therefore refer to explicitly distinguished versions.

8.2. Answers to the Research Questions

8.2.1. RQ1: Executable Reconstruction and Correctness

The available evidence supports an affirmative answer within the evaluated scope. For pre-certified valid inputs, the ordinary-list reconstruction executes the shared paper-facing control flow and independently recovers sorted order from maintained state. Focused tests, bounded exhaustive validation, checked-state audits, and deterministic replay exercise the partial-order and sibling-family invariants. In the formal experiment, all 3600 measured rows belong to 60 exact cases that each passed one untimed checked-state audit before timing. Every row has correct output and no recorded error; all 180 case-algorithm summaries are correct and error-free, and all 60 case audits pass. These records are summarized in section 6.1.

This answer is empirical and bounded. It does not constitute a mathematical proof for every Jordan sequence, nor does it establish recognition of arbitrary inputs. Oracle certification supplies the valid-input precondition, but not the paper core's sorted output.

8.2.2. RQ2: Observed Runtime Behavior

Across the five tested sizes, the median exact-case paper/reference ratio decreases from 3.226 at $n = 32$ to 0.567 at $n = 512$. It is above one through $n = 128$ and below one at $n = 256$ and $n = 512$. The same size ordering and adjacent-size direction occur in the pilot and formal experiments when each run is compared through its own within-case ratios. Python sort has the lowest median call time at every tested size.

These observations answer the question only under the fixed timing scopes. The reference measurement includes its complete oracle-backed pipeline; the paper measurement includes only the pre-certified `minimal` call, with oracle certification and checked diagnostics outside timing. The ratio is therefore not a like-for-like end-to-end speedup. Five tested sizes and an ordinary-list backend also do not establish an asymptotic complexity result. Absolute runtimes from the pilot and formal experiments are not pooled. The matched-case ratios and directional comparison are reported in sections 6.3 and 6.7.

8.2.3. RQ3: Structure and Implementation Cost

The exploratory analysis finds positive within-size associations between reference runtime and both maximum depth and containment-pair density at every tested size. The corresponding paper-runtime

coefficients are smaller and more variable in sign or magnitude. Checked sibling scans, splits, copied items, and transferred items each have positive descriptive associations with minimal paper runtime across the tested sizes. The invariant-check counter is constant within each size, so its within-size Spearman coefficient is undefined rather than zero.

These findings describe the controlled sample; they do not identify causal costs. Structural families are intentionally unbalanced, and checked counters are collected by an untimed policy while runtime comes from the `minimal` policy. The results therefore motivate more targeted experiments but do not show that one structural feature causes a particular runtime change or that the tested generators represent all valid Jordan sequences. The underlying coefficients are reported in section 6.8.

8.3. Contributions in Context

The central contribution is a separation of concerns that the theoretical description can obscure when read as an implementation recipe. The thesis distinguishes:

1. geometric validity and external certification;
2. the incremental sorting control flow and its independent output state;
3. the concrete sibling-list representation and its operation costs; and
4. correctness diagnostics and evidence validation outside the minimal timed path.

This separation supports an executable reconstruction without silently claiming the theoretical backend. It also permits reproducible empirical statements whose provenance and timing boundaries can be checked after the run. The contribution is consequently one of reconstruction, correctness engineering, and bounded empirical characterization. It is not an implementation or experimental demonstration of linear-time Jordan sorting.

8.4. Future Work

Several extensions follow directly from the limitations of the present work. A new experiment must first establish the runtime of the revised ordinary-list core on qualified inputs. A later backend comparison must remeasure both implementations on matched inputs under the same recorded conditions.

First, the now-implemented heterogeneous finger-tree backend needs completion of its representation, locality and cost review, followed by formal matched measurement. It already shares the core and

has qualified inputs and batch calibration. Those advances do not establish the target theoretical bound or isolate storage costs from differing implementation checks and wrappers.

Second, the executable anchor and orientation clarifications could be given a formal proof against an explicit Jordan-sequence model. The present bounded exhaustive validation supplies strong evidence and counterexample-driven regression tests, but a proof would address all valid inputs rather than only enumerated and generated cases.

Third, recognition should be studied through a separate protocol that includes invalid inputs and reason-level correctness. The valid-input sorting evidence reported here cannot answer recognition questions. Error-correcting Jordan sorting and geometric applications such as polygon clipping likewise remain separate extensions.

Finally, future experiments could use larger size ranges, more balanced and more varied case families, equivalent end-to-end pipeline scopes, and multiple recorded runtime environments. These changes would improve empirical reach, but they should retain matched-case ratios, archived manifest-bound evidence, separate evidence-contract validation, and explicit separation between descriptive trends and asymptotic claims. Absolute runtimes from distinct environments should remain separate.

8.5. Closing Perspective

Jordan sorting illustrates why implementing a theoretically efficient algorithm is not merely a matter of transcribing pseudocode. The structural idea, the dynamic representation that realizes its bound, and the evidence needed to evaluate a concrete program are distinct objects. By reconstructing the incremental algorithm with ordinary lists and making those distinctions explicit, this thesis provides a testable bridge between the historical description and an executable research implementation. Its conclusions are deliberately bounded, but within those bounds the reconstruction recovers its own sorted output, is checked by same-core deterministic replay-based state-consistency validation and separate evidence-contract validation, and exhibits a reproducible runtime pattern across the tested cases.

A. Claim Traceability

This appendix records how the thesis-facing claim groups connect to the audited claim inventory and archived, manifest-bound experiment evidence. It is a traceability aid rather than an additional source of empirical results. The historical controlled inventory contains 46 claims: nine implementation claims, thirteen method claims, eleven quantitative result claims, four exploratory claims, and nine required limitations. The version-specific amendments below supplement that historical inventory for the revised implementation.

A.1. Claim Groups and Thesis Locations

Table A.1.: Controlled claim groups and their principal locations in the thesis.

Claim IDs	Principal thesis location	Primary authority class
`I-01--I-03`	sections 4.1 and 4.4	Reference and paper-core output provenance in the implementation source and certified-wrapper boundary.
`I-04--I-05`	sections 4.2 and 4.3	Shared Step 1/2/3 runner, partial-order state, and ordinary-list sibling backend.
`I-06--I-08`	sections 4.4 to 4.5	Diagnostics, deterministic replay, fixed execution policies, and pre-call certification.
`I-09`	section 4.6	Dated ordinary-list scope amendment and missing-theoretical-backend boundary.
`M-01--M-08`	sections 5.1, 5.2.1, 5.3, 5.4 and 5.4.1	Pre-specified formal configuration and the reused timing and scheduling control flow.
`M-09--M-13`	sections 5.4.2, 5.5, 5.6 and 5.6.1	Archived manifest-bound summaries, manifest, evidence-contract validator, and recorded execution environment.
`R-01--R-11`	sections 6.1 to 6.7	Formal-experiment validated evidence and generated analysis tables; the pilot experiment is used only for the directional replication claim.
`E-01--E-04`	section 6.8	Generated structure/runtime and checked-counter relationship tables, interpreted descriptively and non-causally.
`L-01--L-09`	chapter 7	Scope amendment, fixed protocol, timing boundaries, sampling design, and the formal analysis limitations.

The detailed wording, evidence locator, and required boundary for every ID are maintained in `docs/thesis/claim_to_evidence_table.md`. The audited copy has the following SHA-256:

```
cc88363a5aa25d55a56b614b02826aa1
862d7c48c8a1a54bd2c66da17715e2ea
```

The chapter-level audit is recorded in `docs/thesis/final_claim_audit.md`; it verifies coverage of all 46 controlled claims and the cross-chapter interpretation boundaries at its historical baseline. It does not certify the later amendments as part of that earlier audit.

A.2. Implementation Revision and Evidence Scope

The subsequent cost correction is recorded separately in `docs/thesis/cost_revision_audit_20261011.md`. It clarifies copying at both insertion boundaries, always-on membership and ancestor scans, and the limited scope of logical split counters. The lower bound in equation (4.1) is derived from the unchanged source, not from the historical timing measurements. The new resource probes are qualification evidence and do not replace any performance results in this thesis.

The earlier revised implementation account is bound to repository commit `6f74683`. The rollback removal was introduced in commit `337903a`; the full identifiers and source-file hashes are recorded in `docs/thesis/revision_claims_20261011.md`. The formal timing archive instead binds source commit `98868b1b705f6d5f22404ee8ad7b88ad7a834f52`. Its results retain their original version and timing scope.

The earlier account of shared-core access interfaces, separate backend-work records and deterministic replay is bound to repository commit `71e3e71`, with evidence in `docs/validation/shared_core_backend_20261012/`. Replay creates a fresh backend of the same type and configuration and compares logical and physical snapshots. This interface revision does not establish completion or performance of the heterogeneous finger-tree backend.

The amendment to historical claim `I-05` replaces the current implementation's rollback contract with error propagation and mandatory discard of failed state. The clarification to `I-07` distinguishes the unchanged execution-policy switches from the changed recovery work. The ordinary-list complexity limitation in `I-09` remains applicable to that backend. The current tree is implemented and qualified, but a reviewed general efficiency claim and formal matched measurements remain outstanding.

The current implementation and method amendments bind unchanged production sources at `c5b5ac3`. Tree integration, optional detailed work and v2 validation were introduced in `d6622ce`, `84f7e13` and `f77df45`. Qualification executions retain producers `3cafb44` (140 common inputs) and `668371f` (twelve

targeted split inputs). Read-only ID-lifecycle rechecks and actual calibration use `c5b5ac3`; records, all batch attempts, decisions and independent rebuild reports are in `docs/validation/id_lifecycle_calibration_20261015/`. The later archive commit stores those results without becoming the execution version. No formal samples or approved general-proof receipt are claimed.

The revision record links these statements to the current split and insertion code, public-call failure tests, and their thesis locations. The historical method, numerical-result, and exploratory claims keep their original evidence bindings. Their counts, tables, figures, and archive hashes are not updated to represent a new experiment. Future implementations and measurements require their own evidence records.

A.3. Archived Evidence Bindings

Table A.2.: Archived run manifests used by the thesis analysis.

Evidence role	Execution identifier	Manifest SHA-256
Pilot experiment	`week11_pilot_v1__run003`	`ef6a9d17df644eab8c3284d04dc356ce4b68b4efdfc0da7eecb5a9c7136c5141`
Formal experiment	`week12_formal_sorting_v1__run001`	`f42ab0b77cb7ea91ed66c66bc6947a81cd8746b5d2b67d526472260d2b47f850`

Each manifest binds six archive files: raw measurements, case summaries, group summaries, case audits, the fixed configuration, and the recorded runtime environment. The separate formal evidence-contract validator regenerates cases and checked diagnostics, recomputes scheduling and summaries, and verifies these hashes before analysis. The two thesis-facing formal experiment result figures are conversions of the validated SVG analysis outputs. The five explanatory figures are generated from committed TikZ sources. Source and generated hashes for all seven figures are recorded in `thesis/figures/README.md`.

A.4. Interpretation Boundaries

The traceability record preserves the following boundaries across chapters:

- the paper core recovers output from maintained state, while oracle-derived sorted output belongs to the reference pipeline;
- the formal experiment evaluates oracle-certified valid-input sorting, not recognition;
- paper/reference ratios compare different timed pipeline scopes and are not like-for-like end-to-end speedups;

- pilot and formal absolute runtimes are not pooled;
- structure and counter relationships are exploratory and non-causal; and
- the ordinary-list implementation does not implement the historical theoretical backend and carries no linear-time or asymptotic-complexity claim.

Bibliography

Aurenhammer, F. (1987). "Jordan Sorting via Convex Hulls of Certain Non-Simple Polygons". In: *Proceedings of the Third Annual Symposium on Computational Geometry*. SCG '87. New York, NY, USA: Association for Computing Machinery, pp. 21–29. DOI: 10.1145/41958.41961.

Bar-Yehuda, R. and B. Chazelle (1994). "Triangulating Disjoint Jordan Chains". In: *International Journal of Computational Geometry & Applications* 4.4, pp. 475–481. DOI: 10.1142/S0218195994000252.

Berg, M. de, O. Cheong, M. van Kreveld, and M. Overmars (2008). *Computational Geometry: Algorithms and Applications*. 3rd ed. Berlin, Heidelberg: Springer. DOI: 10.1007/978-3-540-77974-2.

Brown, M. R. and R. E. Tarjan (1978). "A Representation for Linear Lists with Movable Fingers". In: *Proceedings of the Tenth Annual ACM Symposium on Theory of Computing*. STOC '78. New York, NY, USA: Association for Computing Machinery, pp. 19–29. DOI: 10.1145/800133.804328.

– (Aug. 1980). "Design and Analysis of a Data Structure for Representing Sorted Lists". In: *SIAM Journal on Computing* 9.3, pp. 594–614. DOI: 10.1137/0209045.

Fung, K. Y., T. M. Nicholl, R. E. Tarjan, and C. J. Van Wyk (June 1990). "Simplified Linear-Time Jordan Sorting and Polygon Clipping". In: *Information Processing Letters* 35.2, pp. 85–92. DOI: 10.1016/0020-0190(90)90111-A.

Hoffmann, K., K. Mehlhorn, P. Rosenstiehl, and R. E. Tarjan (1985). "Sorting Jordan Sequences in Linear Time". In: *Proceedings of the First Annual Symposium on Computational Geometry*. SCG '85. New York, NY, USA: Association for Computing Machinery, pp. 196–203. DOI: 10.1145/323233.323259.

– (1986). "Sorting Jordan Sequences in Linear Time Using Level-Linked Search Trees". In: *Information and Control* 68.1–3, pp. 170–184. DOI: 10.1016/S0019-9958(86)80033-X.

Huddleston, S. and K. Mehlhorn (1982). "A New Data Structure for Representing Sorted Lists". In: *Acta Informatica* 17.2, pp. 157–184. DOI: 10.1007/BF00288968.

Kalibera, T. and R. E. Jones (2013). "Rigorous Benchmarking in Reasonable Time". In: *Proceedings of the 2013 International Symposium on Memory Management*. ISMM '13. New York, NY, USA: Association for Computing Machinery, pp. 63–74. DOI: 10.1145/2464157.2464160.

McGeoch, C. C. (2012). *A Guide to Experimental Algorithmics*. Cambridge: Cambridge University Press. DOI: 10.1017/CBO9780511843747.

Mytkowicz, T., A. Diwan, M. Hauswirth, and P. F. Sweeney (2009). "Producing Wrong Data Without Doing Anything Obviously Wrong!" In: *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems*. ASPLOS XIV. New York, NY, USA: Association for Computing Machinery, pp. 265–276. DOI: 10.1145/1508244.1508275.

Sanders, P. (2009). "Algorithm Engineering: An Attempt at a Definition". In: *Efficient Algorithms: Essays Dedicated to Kurt Mehlhorn on the Occasion of His 60th Birthday*. Ed. by S. Albers, H. Alt, and S. Näher. Vol. 5760. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, pp. 321–340. DOI: 10.1007/978-3-642-03456-5_22.

Sojka, E. (Oct. 1997). "Two Simple and Efficient Algorithms for Jordan Sorting and Polygon Cutting and Clipping". In: *Computer Networks and ISDN Systems* 29.14, pp. 1661–1673. DOI: 10.1016/S0169-7552(97)00081-0.

– (Apr. 1999). "A New Algorithm for Jordan Sorting: Its Average-Case Analysis". In: *Discrete Applied Mathematics* 93.1, pp. 127–140. DOI: 10.1016/S0166-218X(99)00011-6.

Tarjan, R. E. and C. J. Van Wyk (Feb. 1988). "An $O(n \log \log n)$-Time Algorithm for Triangulating a Simple Polygon". In: *SIAM Journal on Computing* 17.1, pp. 143–178. DOI: 10.1137/0217010.